

Angular Questions

Que. Explain Angular Lifecycle Hooks

1. constructor()

- The **first method** that runs when a component is instantiated.
- Used mainly for **dependency injection**.
- Does **not** have access to input properties (@Input values are not available yet).

Example:

```
constructor(private userService: UserService) {  
  console.log('Constructor called!');  
}
```

✔ **Use Case:** Inject services, initialize variables.

✗ **Do Not:** Make HTTP calls (use ngOnInit() instead).

2. ngOnChanges(changes: SimpleChanges)

- Runs **before ngOnInit()** and **whenever input properties change**.
- Executes multiple times if the parent component updates the input property.

Example:

```
@Input() userId: number;
```

```
ngOnChanges(changes: SimpleChanges) {  
  console.log('ngOnChanges called:', changes);  
}
```

✔ **Use Case:** Fetch new data when input changes.

✗ **Do Not:** Use for initialization logic (use ngOnInit()).

3. ngOnInit()

- Called **once** after the component is initialized.
- Ideal for **fetching API data**, setting up event listeners, etc.

Example:

```
ngOnInit() {  
  console.log('ngOnInit called!');  
  this.loadUserData();  
}
```

```
loadUserData() {  
  this.userService.getUser().subscribe(data => this.user = data);  
}
```

✔ **Use Case:** Fetch API data, initialize component state.

✗ **Do Not:** Rely on this for reacting to input property changes (use ngOnChanges()).

4. ngDoCheck()

- Runs **every time** change detection happens (e.g., variable updates, input changes).
- Can be used for **custom change detection logic**.

Example:

```
ngDoCheck() {  
  console.log('ngDoCheck called!');  
}
```

✔ **Use Case:** Detect deep object mutations (when ngOnChanges does not trigger).

✗ **Do Not:** Perform expensive operations (runs very frequently).

5. ngAfterContentInit()

- Runs **once** after projected content (<ng-content>) is initialized.

Example:

```
ngAfterContentInit() {  
  console.log('ngAfterContentInit called!');  
}
```

✔ **Use Case:** Initialize logic related to projected content.

✗ **Do Not:** Modify component's own view (use ngAfterViewInit()).

6. ngAfterContentChecked()

- Runs **after every change detection cycle** when projected content changes.

Example:

```
typescript  
CopyEdit  
ngAfterContentChecked() {  
  console.log('ngAfterContentChecked called!');  
}
```

✔ **Use Case:** Detect changes in <ng-content> dynamically inserted content.

✗ **Do Not:** Modify component properties (can cause infinite loops).

7. ngAfterViewInit()

- Runs **once** after the component and its child views are fully initialized.
- Used for **DOM manipulations, ViewChild initialization**.

Example:

```
@ViewChild('myDiv') myDiv: ElementRef;  
  
ngAfterViewInit() {  
  console.log('ngAfterViewInit called!');  
  console.log(this.myDiv.nativeElement.innerText);  
}
```

✔ **Use Case:** Access @ViewChild or @ViewChildren elements.

✗ **Do Not:** Modify data-bound properties (triggers additional change detection).

8. ngAfterViewChecked()

- Runs **after every change detection cycle** when component's view updates.

Example:

```
ngAfterViewChecked() {  
  console.log('ngAfterViewChecked called!');  
}
```

✓ **Use Case:** Detect changes in dynamically updated DOM elements.

✗ **Do Not:** Perform heavy operations (runs frequently).

9. ngOnDestroy()

- Called **just before** the component is destroyed.
- Used for **cleanup tasks** like **unsubscribing from Observables**.

Example:

```
subscription: Subscription;
```

```
ngOnInit() {  
  this.subscription = this.userService.getUser().subscribe(data => console.log(data));  
}
```

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
  console.log('Component destroyed!');  
}
```

✓ **Use Case:** Unsubscribe from Observables, remove event listeners.

✗ **Do Not:** Perform long-running tasks.

📌 Summary Table

Lifecycle Hook	Runs When?	Use Case
constructor()	On component creation	Inject services, initialize variables
ngOnChanges()	When @Input property changes	React to parent input changes
ngOnInit()	After component initializes	Fetch API data, initialize logic
ngDoCheck()	On every change detection	Detect deep object mutations
ngAfterContentInit()	After <ng-content> initializes	Initialize projected content
ngAfterContentChecked()	After every change detection affecting <ng-content>	Detect changes in projected content
ngAfterViewInit()	After component & child views initialize	Access @ViewChild elements

Lifecycle Hook	Runs When?	Use Case
<code>ngAfterViewChecked()</code>	After every change detection affecting view	Detect updates in dynamically added elements
<code>ngOnDestroy()</code>	Before component destruction	Cleanup, unsubscribe from Observables

Key Takeaways

1. Use `ngOnInit()` for initialization (fetching data, API calls).
2. Use `ngOnDestroy()` for cleanup (unsubscribe, remove listeners).
3. Use `ngOnChanges()` to detect changes in `@Input` properties.
4. Use `ngAfterViewInit()` to access DOM elements with `@ViewChild`.
5. Avoid heavy computations in `ngDoCheck()` and `ngAfterViewChecked()` to prevent performance issues.

Que. Difference Between `ngDoCheck` and `ngOnChanges`

Feature	<code>ngOnChanges</code>	<code>ngDoCheck</code>
When it Runs	Runs when <code>@Input()</code> property changes	Runs on every change detection cycle
Trigger	Only triggered when Angular detects changes in input properties	Can detect deeper changes that <code>ngOnChanges</code> might miss
Use Case	Used for tracking changes in <code>@Input()</code> values	Used for custom change detection logic
Performance	More efficient (runs only when input changes)	Less efficient (runs frequently)

Que. Explain memory leakage.

Memory leaks in Angular occur when unused objects, DOM elements, or subscriptions remain in memory.

Examples: Unsubscribed observables, unremoved event listeners, or detached DOM nodes.

Que. How to Detect Memory Leaks in Angular

1. Browser DevTools:

Use the Performance tab in Chrome DevTools: Look for memory that increases continuously and never reduces.

Take Heap Snapshots in the Memory tab: Look for objects that remain in memory even after the component or service is no longer in use.

2. Profiler:

Use libraries like Augury for profiling Angular applications.

Que. How to Avoid Memory Leaks in Angular

1. Unsubscribe from Observables:

Always unsubscribe from subscriptions in `ngOnDestroy` or use the **takeUntil** pattern with `Subject`.

```
private destroy$ = new Subject<void>();
ngOnInit() {
  this.http.get('api/data')
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => console.log(data));
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

2. Use Async Pipe:

Prefer the async pipe in templates instead of manually subscribing to observables.

```
<div *ngIf="data$ | async as data">
  {{ data }}
</div>
```

3. Remove Event Listeners:

Detach global event listeners in `ngOnDestroy`

```
ngOnInit() {
  window.addEventListener('resize', this.onResize);
}

ngOnDestroy() {
  window.removeEventListener('resize', this.onResize);
}
```

4. Use TrackBy in ngFor:

Use the `trackBy` function to optimize DOM rendering and prevent unnecessary memory allocation.

```
<div *ngFor="let item of items; trackBy: trackByFn">
  {{ item.name }}
</div>
```

```
trackByFn(index: number, item: any) {  
  return item.id; // Unique identifier  
}
```

Que. How to Handle Large-Scale Data in Angular Tables

1. Virtual Scrolling

Explanation: Instead of rendering all rows at once, virtual scrolling only renders the rows currently visible in the viewport. As the user scrolls, rows are dynamically added or removed from the DOM.

2. Pagination

Explanation: Divide the data into smaller chunks (pages) and display one page at a time.

3. trackBy

if the array is updated (e.g., new items added or modified), Angular re-renders the entire list, which can lead to performance issues for large datasets.

Que. What is OnPush?

OnPush is a change detection strategy in Angular that improves application performance by limiting the number of checks Angular performs on components. It ensures that a component's view is only updated when its input properties change or when an event is triggered explicitly.

Default Change Detection Strategy

- The default strategy is **Default**.
- Angular checks the entire component tree, even if some components are not affected by the changes.
- This can cause performance bottlenecks in applications with a large number of components.

What is OnPush?

The **OnPush** change detection strategy tells Angular to skip checking a component unless:

1. **Input Properties Change:** If any of the @Input() properties are updated (via reference or value).
2. **Events Triggered Internally:** If an event in the component (e.g., a click or form submission) updates the state.
3. **Manual Trigger:** If you manually trigger change detection using methods like `markForCheck()` or `detectChanges()` from `ChangeDetectorRef`.

Example:

```
import { Component, ChangeDetectionStrategy, Input } from '@angular/core';

@Component({
  selector: 'app-onpush-demo',
  template: `
    <h3>{{ data.name }}</h3>
    <button (click)="updateData()">Update Data</button>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush, // Use OnPush strategy
})
export class OnPushDemoComponent {
  @Input() data: { name: string };

  updateData() {
    this.data.name = 'Updated Name'; // Won't trigger view update unless reference changes
  }
}
```

When to Use

- **Large Applications:** With many components and frequent updates, OnPush improves performance.
- **Data Immutability:** Works best when using immutable data structures (e.g., new references for updated objects).

Common Pitfalls

1. **Direct Mutations:** If you directly modify an object or array without changing its reference, the view won't update:
 // Does NOT trigger update with OnPush
 this.data.name = 'Updated Name';
2. **Event Handling:** Changes outside the component or parent hierarchy won't trigger updates unless manually handled.

Que. Problem Statement

When you update something in the parent component, Angular runs change detection for the entire component tree, including the child component, which may cause performance issues if the child component contains heavy data (e.g., a table with 1000 lines).

Solution

1. Use OnPush Change Detection Strategy in the Child Component

The OnPush strategy ensures that Angular runs change detection for the child component only when:

- An @Input() property reference changes.
- An event occurs inside the child component.
- Change detection is triggered manually.

```
import { Component, ChangeDetectionStrategy, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <table>
      <tr *ngFor="let row of data">
        <td>{{ row.id }}</td>
        <td>{{ row.name }}</td>
      </tr>
    </table>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush, // Use OnPush strategy
})
export class ChildComponent {
  @Input() data: any[] = [];
}
```

2. Detach Change Detection in the Child

If the child component data is entirely static or infrequently updated, you can detach change detection in the child to prevent it from running unnecessarily.

```
import { Component, ChangeDetectorRef } from '@angular/core';
```

```
@Component({
  selector: 'app-child',
  template: `
    <table>
      <tr *ngFor="let row of data">
        <td>{{ row.id }}</td>
        <td>{{ row.name }}</td>
      </tr>
    </table>
  `,
})
export class ChildComponent {
  data = Array.from({ length: 1000 }, (_, i) => ({ id: i + 1, name: `Row ${i + 1}` }));

  constructor(private cdr: ChangeDetectorRef) {
    // Detach change detection
    this.cdr.detach();
  }

  updateTableData(newData: any[]) {
    this.data = newData;
  }
}
```

```
this.cdr.detectChanges(); // Manually trigger change detection
}
}
```

3. Use trackBy with *ngFor in the Child Component

If the table data in the child component is dynamically updated, use the trackBy function to optimize rendering.

html

```
<tr *ngFor="let row of data; trackBy: trackById">
  <td>{{ row.id }}</td>
  <td>{{ row.name }}</td>
</tr>
```

```
trackById(index: number, item: any): number {
  return item.id; // Use a unique identifier
}
```

Que. What is DOM sanitization

DOM sanitization in Angular is a security feature that cleans and validates untrusted content before rendering it in the DOM. It prevents attacks like **Cross-Site Scripting (XSS)** by removing harmful code such as scripts, malicious URLs, or unsafe styles.

How to Use DomSanitizer in Common Scenarios

```
import { DomSanitizer } from '@angular/platform-browser';
this.safeHtmlContent = this.sanitizer.bypassSecurityTrustHtml('<p>Trusted Content</p>');
this.safeUrl = this.sanitizer.bypassSecurityTrustUrl('https://trusted-website.com');
this.safeStyle = this.sanitizer.bypassSecurityTrustStyle('color: red;');
```

Que. How to check a third-party library for vulnerabilities

- Install dependencies: npm install
- Run the audit: npm audit
- Review the report: The report will show vulnerabilities in your dependencies, with severity levels and possible fixes.
- Automatically fix issues (if possible): npm audit fix
- Manually fix remaining issues by updating versions or replacing vulnerable libraries.

Que. What is standalone component

A **standalone component** in Angular is a component that can function independently, without being part of an Angular module (NgModule). This feature, introduced in **Angular 14**, simplifies the way components are used and makes it easier to manage smaller, self-contained, reusable parts of your application.

What Makes Standalone Components Different?

In traditional Angular applications, every component must be declared in an NgModule. Components, directives, and pipes need to be part of a module before they can be used in the application. This makes managing the application more complex, especially as the application grows.

Standalone components, however, break this rule by allowing a component to be completely self-contained. You can use them without having to declare them inside a module. This makes it easier to use components in routing, other components, or external libraries.

Key Features of Standalone Components:

1. No Need for an NgModule Declaration:

- Traditional Angular components are declared inside an NgModule. However, a standalone component can be created without needing to be declared in any module.
- This means that the component can directly be imported and used in other parts of the app.

2. Self-contained Components:

- A standalone component brings with it everything it needs: template, styles, imports, and even dependencies like directives or pipes. It doesn't rely on the module system for these.

3. Direct Use in Routing:

- You can directly use standalone components in routing configurations, eliminating the need to declare them in the routing module.

4. Reduced Boilerplate:

- Standalone components help reduce the amount of boilerplate code (modules) for simple components that don't require complex configuration or dependencies.

5. Improved Reusability:

- Since a standalone component doesn't need a module, it's easier to reuse across applications without the extra configuration steps.

Que. What is <router-outlet>?

The `<router-outlet>` directive in Angular is used as a placeholder for dynamic routing. It acts as a container where the routed component (based on the current route) will be displayed. When a route is activated, Angular will replace the `<router-outlet>` with the corresponding component's template.

Key Uses of <router-outlet>:

1. Displaying Routed Components:

The main purpose of <router-outlet> is to display components based on the active route. When you navigate to a specific route, Angular injects the component defined for that route inside the <router-outlet>.

```
<!-- In your app.component.html -->
<h1>Welcome to the App!</h1>
<router-outlet></router-outlet> <!-- Routed components will be displayed here -->
```

2. Nested Routes (Child Routes):

When you define child routes (nested routes), you can place another <router-outlet> in the parent component's template to display the child components.

```
<!-- Parent component template -->
<h2>Parent Component</h2>
<router-outlet></router-outlet> <!-- Child routes will be rendered here -->
```

For instance, if the route is /parent/child, the child component will be displayed inside the router-outlet of the parent component.

3. Multiple Router Outlets (Named Outlets):

You can use **multiple outlets** in your application. Each outlet can be named, and different components can be rendered into different outlets. This is useful for situations where you have multiple sections in your app that require different routing contexts (like sidebars, modals, or multiple views).

```
<router-outlet name="primary"></router-outlet> <!-- Primary route content -->
<router-outlet name="sidebar"></router-outlet> <!-- Sidebar content -->
```

In the routing configuration, you specify the outlet name:

```
const routes: Routes = [
  {
    path: 'main',
    component: MainComponent,
    outlet: 'primary' // Content will be rendered in 'primary' outlet
  },
  {
    path: 'sidebar',
    component: SidebarComponent,
    outlet: 'sidebar' // Sidebar content will be rendered in 'sidebar' outlet
  }
];
```

4. Lazy Loading:

`<router-outlet>` is also used in **lazy-loaded modules**. When a route is accessed that requires a module to be loaded on demand, the component of that module will be displayed inside the router-outlet.

```
const routes: Routes = [  
  {  
    path: 'lazy',  
    loadChildren: () => import('./lazy/lazy.module').then(m => m.LazyModule)  
  }  
];
```

Inside the lazy-loaded module's routing, you'll have the usual `<router-outlet>` to load the components for the specific route.

Basic Example of router-outlet with Routes:

1.App Component (app.component.html):

```
<h1>Angular Routing Example</h1>  
<nav>  
  <a routerLink="/home">Home</a>  
  <a routerLink="/about">About</a>  
</nav>
```

```
<router-outlet></router-outlet> <!-- Routed component will be displayed here -->
```

2.Routing Configuration (app-routing.module.ts):1

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'about', component: AboutComponent },  
  { path: '', redirectTo: '/home', pathMatch: 'full' }  
];
```

Que. What is @HostListener and @HostBinding

In Angular, **@HostListener** and **@HostBinding** are decorators used to interact with the **host element** (the DOM element the directive or component is applied to).

1. @HostListener

- **Purpose:** Allows you to listen to events on the host element and execute a method when the event occurs.
- **Use Case:** Used to handle user interactions like clicks, mouse events, keypresses, etc.

Syntax: `@HostListener(eventName: string, args?: string[])`

eventName: The event to listen to (e.g., 'click', 'mouseover').

args: Optional parameters (like `$event`) passed to the method.

Example:

```
import { Directive, HostListener } from '@angular/core';
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostListener('mouseenter') onMouseEnter() {
    console.log('Mouse entered the element!');
  }
  @HostListener('mouseleave') onMouseLeave() {
    console.log('Mouse left the element!');
  }
}
```

2. @HostBinding

- **Purpose:** Binds a property or attribute of the host element to a directive or component's property.
- **Use Case:** Dynamically set classes, styles, or other attributes on the host element.

Syntax: @HostBinding(bindingPropertyName: string)

bindingPropertyName: The DOM property or attribute (like class, style, attr.someAttribute, etc.).

Example:

```
import { Directive, HostBinding } from '@angular/core';
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostBinding('style.backgroundColor') backgroundColor: string;
  constructor() {
    this.backgroundColor = 'yellow'; // Sets initial background color
  }
}
```

Combining @HostListener and @HostBinding

```
import { Directive, HostBinding, HostListener } from '@angular/core';
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostBinding('style.backgroundColor') backgroundColor: string;
  @HostListener('mouseenter') onMouseEnter() {
```

```

    this.backgroundColor = 'lightblue';
  }
  @HostListener('mouseleave') onMouseLeave() {
    this.backgroundColor = 'white';
  }
}

```

Que. What is an Interceptor in Angular?

An **Interceptor** in Angular is a service that allows you to intercept and manipulate **HTTP requests and responses** made through Angular's HttpClient. It acts like middleware and can handle tasks such as adding headers, logging requests, handling errors, and modifying responses **before the request is sent** to the server or **after the response is received**.

Why Use an Interceptor?

Interceptors provide a centralized and reusable way to handle:

Use of Interceptors

1. Add or Modify Headers

- Example: Adding authentication tokens (e.g., Authorization header).

2. Handle Errors Globally

- Centralize error handling logic for all HTTP requests.

3. Log HTTP Requests/Responses

- Useful for debugging or logging purposes.

4. Show/Hide Loading Indicators

- Manage the visibility of spinners or loaders based on request states.

5. Retry Failed Requests

- Automatically retry requests in case of network issues.

Example: Adding Authorization Headers

1.Create an Interceptor Run the following command to generate an interceptor:
ng generate interceptor auth

2.Write the Code In the generated interceptor file (e.g., auth.interceptor.ts):

```

import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from
 '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('authToken'); // Retrieve the token
    if (token) {
      // Clone the request and add the Authorization header
      const clonedReq = req.clone({

```

```

    setHeaders: {
      Authorization: `Bearer ${token}`
    }
  });
  return next.handle(clonedReq); // Pass the modified request
}
return next.handle(req); // Pass the original request if no token is found
}
}

```

3.Register the Interceptor Add the interceptor to your AppModule:

```

import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { AuthInterceptor } from './auth.interceptor';

```

```

@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true // Allows multiple interceptors
    }
  ]
})
export class AppModule {}

```

Note: For angular 18 In your main.ts, you can configure the interceptors within the providers array when bootstrapping the root component.

```

import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { AppComponent } from './app.component';
import { AuthInterceptor } from './auth.interceptor';
import { LoggingInterceptor } from './logging.interceptor';

```

```

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(
      withInterceptors([
        AuthInterceptor,
        LoggingInterceptor
      ])
    )
  ]
}).catch(err => console.error(err));

```

Que. I have total 10 API, I want to use interceptor only for 7 API and skip for 3 API how can I do this.

To use an interceptor for only a subset of APIs in Angular, you can implement logic inside the intercept method to conditionally skip or include specific APIs. Here's how you can do it:

```
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from
 '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    // List of APIs to exclude from the interceptor
    const excludedUrls = [
      '/api/skip1', // Example API endpoint
      '/api/skip2',
      '/api/skip3'
    ];

    // Check if the current request URL matches any excluded URL
    const shouldSkip = excludedUrls.some(url => req.url.includes(url));

    if (shouldSkip) {
      // If the request matches an excluded URL, pass it through without modification
      return next.handle(req);
    }

    // If not excluded, apply the interceptor logic
    const token = 'YOUR_AUTH_TOKEN';
    const clonedReq = req.clone({
      headers: {
        Authorization: `Bearer ${token}`
      }
    });

    return next.handle(clonedReq);
  }
}
```

Que. How can you catch error get from backend?

```
fetchData() { this.dataService.getData('https://example.com/api/data').  
subscribe({ next: (response) => { this.data = response; // Assign the data to the component  
this.errorMessage = ""; // Clear error message },  
error: (error) => { this.errorMessage = error.message; // Show error message this.data = null;  
// Clear data }, }); }
```

Que. What is used of ng model?

In Angular, ngModel is a directive used for two-way data binding in template-driven forms. It synchronizes the data between the HTML view (template) and the TypeScript component.

```
<form>  
<label for="email">Email:</label>  
<input  
  type="email"  
  id="email"  
  name="email"  
  [(ngModel)]="email"  
  required  
>  
<p *ngIf="!email">Email is required</p>  
</form>
```

Que. Difference between Template driven form and reactive form

Feature	Template-Driven Forms	Reactive Forms
Form Definition	Defined in the template with directives like <code>ngModel</code>	Defined programmatically in the component class
Binding	Uses <code>[(ngModel)]</code> for two-way data binding	Uses <code>formControlName</code> and <code>formGroup</code> for form control and binding
Validation	Validation is defined in the template (<code>required</code> , <code>minlength</code> , etc.)	Validation is defined programmatically with <code>Validators</code>
Control Over Form	Limited control, based on the form's HTML structure	Full control over form, its validation, and dynamics
Scalability	Good for small forms	Better for complex and large forms
Testing	Harder to test due to implicit logic	Easier to test due to explicit form control setup
State Management	Less explicit, Angular automatically handles state	Explicit management of state, form controls, and value changes
Dynamic Forms	Not as flexible for dynamic forms	Very flexible for dynamic and complex forms

Que. What is track by?

In Angular, **trackBy** is a directive used in **ngFor** loops to improve performance by helping Angular track and identify individual items in a list when the list changes. It prevents unnecessary re-rendering of list items when only a few items have changed, thus improving the overall performance, especially in large lists.

By default, Angular tracks list items by their **identity** (i.e., the object reference). If the list changes, Angular will compare the entire list and re-render all the items. However, if you provide a custom **trackBy** function, Angular will use it to track items based on a unique identifier, reducing the number of DOM updates.

OR

When you use **ngFor** to iterate over a list and render elements, Angular needs to keep track of each item for change detection. By default, Angular tracks list items by their object identity. This means if you modify an array (e.g., adding, removing, or reordering items), Angular will re-render the entire list because it doesn't know which item corresponds to which DOM element.

The track by clause allows you to specify a unique identifier for each item in your list. This way, Angular can efficiently track which items have changed and only re-render those specific elements instead of the whole list.

```
//template
```

```
<div *ngFor="let item of items; trackBy: trackByFn">
```

```
  {{ item.name }}
</div>
```

```
//component
```

```
trackByFn(index: number, item: any): number { return item.id; // Use 'id' as unique identifier
}
```

What is difference between ng build and ng serve?

1. ng serve

- **Purpose:** The ng serve command is used to build and serve your Angular application in a local development environment.
- **Functionality:**
 - It **compiles** the application, **starts a development server**, and **watches for changes** in your source files.
 - It serves the app at a local development URL (usually `http://localhost:4200`), where you can see your application running.

2. ng build

- **Purpose:** The ng build command is used to **compile the Angular application** and **create a distributable version** of the app.
- **Functionality:**
 - It compiles the application and places the output in the **dist/** folder (by default).
 - It generates static files (HTML, CSS, JavaScript) that are optimized and ready for deployment.

- It runs in **production mode** (if you specify the `--prod` flag) to perform optimizations such as minification, dead code elimination, and tree shaking, making the app ready for deployment.

Que. I want to load some config before starting angular app how can I do?

OR Explain APP_INITIALIZER

Steps to Load Configuration Before Starting Angular App

1. **Create a Config Service:** Create a service that loads the configuration from an external file or API.
2. **Use APP_INITIALIZER:** In your Angular module (AppModule providers), use APP_INITIALIZER to delay the application startup until the configuration is loaded.
3. **Return a Promise:** The APP_INITIALIZER provider expects a function that returns a **Promise** or **Observable**. Angular will wait for the promise to resolve before bootstrapping the application.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';
import { AppComponent } from './app.component';
import { ConfigService } from './config.service';
import { APP_INITIALIZER } from '@angular/core';
```

```
export function loadConfig(configService: ConfigService) {
  return () => configService.loadConfig().toPromise();
}
```

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, HttpClientModule],
  providers: [
    {
      provide: APP_INITIALIZER,
      useFactory: loadConfig,
      deps: [ConfigService],
      multi: true,
    },
  ],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

Que. What is an AuthGuard in Angular?

An **AuthGuard** in Angular is a service that is used to **prevent unauthorized users** from accessing certain routes or pages in an Angular application. It is a **route guard** that checks if a user is authenticated or authorized before allowing navigation to a protected route.

How it Works

When a route is requested, Angular will check for any guards that are defined on that route. If an **AuthGuard** is present, **canActivate()** method is called when a user tries to access a route. It checks if the user is authenticated. It will execute the logic in the guard and either allow the navigation or cancel it based on the authentication state.

Purpose of an AuthGuard

- **Protect routes:** Ensures that only authorized users can access specific routes in your Angular app (e.g., user profile, admin dashboard).
- **Redirect unauthorized users:** If the user is not authenticated, the guard can redirect them to a login page or a different route.

Steps to Implement an AuthGuard

1. **Create the AuthGuard service.**
2. **Define the AuthGuard logic** to check the authentication status.
3. **Apply the AuthGuard to routes** that you want to protect.

1.Create the AuthGuard Service:

CMD:ng generate guard auth

```
auth.guard.ts
import { Injectable } from '@angular/core';
import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot, Router } from '@angular/router';
import { Observable } from 'rxjs';
import { AuthService } from './auth.service'; // Your Auth service
```

```
@Injectable({
  providedIn: 'root',
})
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}
```

```
  canActivate(
    next: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): Observable<boolean> | Promise<boolean> | boolean {
    // Check if the user is authenticated
    const isAuthenticated = this.authService.isAuthenticated();
```

```

    if (isAuthenticated) {
      return true;
    } else {
      // If not authenticated, redirect to login page
      this.router.navigate(['/login']);
      return false;
    }
  }
}

```

2. Define Authentication Logic in AuthService

```

auth.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  constructor() {}

  // Method to check if the user is authenticated
  isAuthenticated(): boolean {
    const token = localStorage.getItem('auth_token');
    return !!token; // If a token exists, the user is authenticated
  }

  // Method to log in the user
  login(token: string): void {
    localStorage.setItem('auth_token', token);
  }

  // Method to log out the user
  logout(): void {
    localStorage.removeItem('auth_token');
  }
}

```

3. Apply the AuthGuard to Routes

```

app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { AuthGuard } from './auth.guard';
import { HomeComponent } from './home/home.component';
import { DashboardComponent } from './dashboard/dashboard.component';

```

```

import { LoginComponent } from './login/login.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard', // Protected route
    component: DashboardComponent,
    canActivate: [AuthGuard] // Apply the AuthGuard here
  },
  { path: '**', redirectTo: '' }, // Catch-all route
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule],
})
export class AppRoutingModule {}

```

Que. What is a Route Guard in Angular?

A **Route Guard** in Angular is a mechanism used to control navigation to and from routes in an Angular application. It acts as a **gatekeeper** that decides whether a user is allowed to **activate, deactivate, or resolve** a particular route.

Route guards can be used to enforce various conditions such as **authentication, authorization, unsaved changes**, and **data fetching** before or after navigating to a route.

Types of Route Guards in Angular

Angular provides several types of route guards, each serving a specific purpose. They all implement one or more of the following interfaces:

1. **CanActivate**: Determines whether a route can be activated (used before entering the route). Eg. `AuthGuard`, `routeGuard`
2. **CanActivateChild**: Determines whether a child route can be activated (used before entering a child route).
3. **CanDeactivate**: Determines whether the user can leave the current route (used to prevent leaving a route with unsaved changes).
4. **Resolve**: Fetches data before the route is activated.
5. **CanLoad**: Prevents the route from being loaded (used for lazy-loaded modules).

Que. How to determine whether a user is logged in or not?

We can check if the user is logged in by checking if the JWT token exists in **localStorage** or **sessionStorage**.



Que. What is used of --force while installing packages

The --force flag in package management (such as when using **npm** or **yarn**) is used to force the installation of packages, even if there are warnings, conflicts, or issues related to package versions or dependencies. This flag bypasses certain checks or restrictions that would normally prevent the installation from proceeding.

Que. What is the use of @ViewChild

@ViewChild is a commonly used decorator in Angular. It allows you to access a child component, directive, or DOM element within the parent component.

@ViewChild is used to:

- Access child components, directives, or DOM elements within the same view.
- Interact with them programmatically, such as calling their methods, accessing properties, or manipulating the DOM.

How @ViewChild Works

- It captures the first matching element/component in the component's view (DOM). You can use @ViewChild after Angular's view is initialized, typically inside ngAfterViewInit() lifecycle hook.

```
<!-- parent.component.html -->
<app-child #childComponent></app-child>
<button (click)="callChildMethod()">Call Child Method</button>
```

```
// parent.component.ts
import { Component, ViewChild } from '@angular/core';
import { ChildComponent } from './child.component';
```

```
@Component({
  selector: 'app-parent',
  templateUrl: './parent.component.html',
})
export class ParentComponent {
  @ViewChild('childComponent') childComponent!: ChildComponent;

  callChildMethod() {
    this.childComponent.someMethod(); // Call method in child component
  }
}
```

Que. Give me example where you have invoked manual change detection and how

Manual change detection is needed when Angular does not detect changes automatically, such as when working with code outside Angular's zone (e.g., setTimeout, setInterval or third-party libraries).

```
import { Component, ChangeDetectorRef, OnInit } from '@angular/core';
```

```
@Component({
  selector: 'app-timer',
  template: `<p>Time left: {{ timer }} seconds</p>`,
})
export class TimerComponent implements OnInit {
  timer: number = 10;

  constructor(private cdr: ChangeDetectorRef) {}

  ngOnInit() {
    setInterval(() => {
      this.timer--;
      this.cdr.detectChanges(); // Manually update the view
    }, 1000);
  }
}
```

Key Difference:

- **markForCheck():** Marks the component and its ancestors for checking in the next change detection cycle. It defers the update to the regular Angular change detection flow.
- **detectChanges():** Immediately runs change detection for the current component and its children.

Que. How to add bootstrap in application

Steps to Add Bootstrap

1. Install Bootstrap via npm

Run the following command in your Angular project's root directory: `npm install bootstrap`
This will download Bootstrap and add it to your project's dependencies.

2. Include Bootstrap CSS in angular.json

To apply Bootstrap styles globally, include its CSS file in the angular.json file. Locate the styles array in the angular.json file and add the Bootstrap CSS file:

```
"styles": [
  "src/styles.css",
  "node_modules/bootstrap/dist/css/bootstrap.min.css"
],
```

If you use SCSS, replace styles.css with styles.scss.

3. Optionally Include Bootstrap JS

If you need Bootstrap's JavaScript features (like modals, tooltips, or dropdowns), add the JS files and dependencies to the angular.json file.

First, install Popper.js and Bootstrap JavaScript:

```
npm install @popperjs/core
```

Then add the JS files in the scripts array of angular.json:

```
"scripts": [  
  "node_modules/@popperjs/core/dist/umd/popper.min.js",  
  "node_modules/bootstrap/dist/js/bootstrap.min.js"  
],
```

Que. What is async pipe?

The **async pipe** in Angular is a built-in pipe that simplifies the subscription to and handling of **observables** or **promises** directly in templates. It automatically subscribes to the data stream, retrieves the emitted value, and unsubscribes when the component is destroyed.

Key Features:

1. Automatically manages subscriptions, eliminating the need to manually subscribe and unsubscribe.
2. Prevents memory leaks by automatically unsubscribing when the component is destroyed.
3. Used in **templates** for cleaner and more concise code.

Syntax: {{ observableData\$ | async }}

Eg.

```
import { Component } from '@angular/core';
```

```
import { Observable, of } from 'rxjs';
```

```
@Component({
```

```
  selector: 'app-example',
```

```
  template: `
```

```
    <h1>Async Pipe Example</h1>
```

```
    <p>Data: {{ data$ | async }}</p>
```

```
  `,
```

```
})
```

```
export class ExampleComponent {
```

```
  data$: Observable<string> = of('Hello, Async Pipe!');
```

```
}
```

Here:

- data\$ is an observable emitting the string "Hello, Async Pipe!".
- The **async pipe** subscribes to data\$, retrieves the emitted value, and displays it in the template.

Drawbacks:

1. Limited Reusability: Using the **AsyncPipe** multiple times in a template creates separate subscriptions for the same observable, which can lead to inefficiency.

2. **Performance Overhead:** Frequent change detection cycles can be triggered if the observable emits too often because it is pipe, potentially affecting performance.
3. **No Manual Control:** You lose control over subscription management since it automatically subscribes and unsubscribes.

Que. How can we say AsyncPipe is type of pure pipe

The **AsyncPipe** in Angular is a **pure pipe** because it does not change its output unless its input observable or promise changes.

Que. How to apply background colors to rows in a table based on the age condition(age>18 blue else green)

```
<table border="1" style="width: 50%; text-align: left;">
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let person of people" [ngStyle]="{'background-color': person.age < 18 ? 'blue'
: 'green', 'color': 'white'}">
      <td>{{ person.name }}</td>
      <td>{{ person.age }}</td>
    </tr>
  </tbody>
</table>
```

Que. How to apply alternate row colors (white and gray)

```
<table border="1" style="width: 50%; text-align: left;">
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    <tr
      *ngFor="let person of people; let i = index"
      [ngClass]="{
        'blue-bg': person.age < 18,
        'green-bg': person.age >= 18,
        'even-row': i % 2 === 0,
        'odd-row': i % 2 !== 0
      }">
```

```

    >
    <td>{{ person.name }}</td>
    <td>{{ person.age }}</td>
  </tr>
</tbody>
</table>

```

OR Using CSS nth-child

```

/* Alternating row colors using :nth-child */
tbody tr:nth-child(odd) { background-color: lightgray; }
tbody tr:nth-child(even) { background-color: white; }

```

OR Using a Custom Directive

```

import { Directive, ElementRef, Renderer2, Input, OnInit } from '@angular/core';

```

```

@Directive({
  selector: '[appAlternatingRow]'
})

```

```

export class AlternatingRowDirective implements OnInit {

```

```

  @Input() appAlternatingRow: number;

```

```

  constructor(private el: ElementRef, private renderer: Renderer2) {}

```

```

  ngOnInit() {

```

```

    if (this.appAlternatingRow % 2 === 0) {

```

```

      this.renderer.setStyle(this.el.nativeElement, 'background-color', 'white');

```

```

    } else {

```

```

      this.renderer.setStyle(this.el.nativeElement, 'background-color', 'lightgray');

```

```

    }

```

```

  }

```

```

}

```

```

<tbody>

```

```

<tr *ngFor="let person of people; let i = index" [appAlternatingRow]="i">

```

```

  <td>{{ person.name }}</td>

```

```

  <td>{{ person.age }}</td>

```

```

</tr>

```

```

</tbody>

```

Que. What is CORS origin error, how can we handle it?

CORS (Cross-Origin Resource Sharing) is a security mechanism implemented by browsers to restrict web pages from making requests to a domain different from the one that served the web page.

When you try to access an API or resource hosted on a different domain, the browser checks if the server allows such requests by looking for specific headers. If the server doesn't allow cross-origin requests, the browser blocks the request and throws a CORS error.

- **For Production:** The best way to handle CORS is to **configure the server** to allow requests from trusted origins.
- **For Development:** If you can't modify the backend, you can use **Angular proxy** or server-side proxy in other frameworks to avoid CORS issues.
- **For Development:** start chrome --disable-web-security --user-data-dir="C:/chrome_dev"

Que. What is ng container?

<ng-container> is a special Angular directive that acts as a placeholder without rendering any actual DOM element. It is used to group multiple elements or directives in Angular templates without introducing extra HTML elements in the DOM.

Use Cases:

1. Grouping Elements without Adding Extra DOM Nodes:

Example: Without ng-container, using *ngIf would require an extra <div>, but with ng-container, you don't need that extra wrapper.

```
<ng-container *ngIf="isVisible">
  <h1>Visible</h1>
  <p>Some content</p>
</ng-container>
```

In this example, if isVisible is true, both <h1> and <p> will be rendered, but no <ng-container> will be added to the DOM.

2. Nested Structural Directives:

You can use ng-container to group elements when you have multiple structural directives on the same block.

```
<ng-container *ngIf="isVisible">
  <div *ngFor="let item of items">
    <p>{{ item.name }}</p>
  </div>
</ng-container>
```

Here, ng-container allows you to conditionally render the entire block (using *ngIf) and then loop through the items with *ngFor

Summary:

<ng-container> is useful when you need to group elements in a template without adding any extra DOM nodes, especially when using structural directives. It keeps your template clean and avoids unnecessary wrapper elements in the DOM.

Que. In same div can we use two structural directives ? Why?

cannot apply two structural directives (like `*ngIf`, `*ngFor`, etc.) to the same DOM element (e.g., `<div>`), because structural directives internally modify the DOM and add/remove elements, which causes a conflict if more than one is applied to the same element.

To work around this limitation, you can wrap the structural directives in an `<ng-container>`. Since `<ng-container>` is not an actual DOM element, it only exists in the Angular template and doesn't interfere with the DOM structure. This allows you to use multiple structural directives.

Que. What is ng template?

`<ng-template>` is a special Angular directive that defines a **template fragment** that can be conditionally included or rendered in the DOM. It allows you to declare parts of your template that are not immediately rendered by default, but can be used or rendered later in the view based on conditions or logic.

Key Features of `<ng-template>`:

- It **does not** render anything to the DOM directly.
- It acts as a **template container** that is used by Angular's structural directives (like `*ngIf`, `*ngFor`, etc.) to conditionally render content.
- It can be used with structural directives, custom directives, or even for dynamic content rendering.

Common Use Cases:

1. Using with Structural Directives

```
<ng-template [ngIf]="isVisible">
  <p>This content is visible when 'isVisible' is true</p>
</ng-template>
```

2. Using with `*ngFor`:

```
<ul>
  <li *ngFor="let item of items; let i = index">
    <ng-template>
      <span>{{ i + 1 }}. {{ item }}</span>
    </ng-template>
  </li>
</ul>
```

3. Dynamic Content Rendering (via `ngTemplateOutlet`)

```
<ng-template #contentTemplate>
  <div>This is a dynamic content template</div>
</ng-template>
```

```
<div *ngIf="showContent">
  <ng-container [ngTemplateOutlet]="contentTemplate"></ng-container>
</div>
```

Que. What is content project (ng content)?

<ng-content> is a directive in Angular that enables **content projection**. It allows you to insert content from a parent component into a child component. Essentially, **content projection** allows you to define a placeholder in your child component where the parent component can inject HTML content.

This is particularly useful when you want to create **reusable components** and allow the parent to pass dynamic content to the child component, without hardcoding the content into the child.

Eg.

<!--app.component.html (Parent) →

<app-child>

 <p>This is the projected content!</p>

</app-child>

<!--child.component.html (Child) →

<div class="child-component">

 <h2>Child Component</h2>

 <ng-content></ng-content> <!--This is where the parent's content will be projected →

</div>

<!--output --!>

<div class="child-component">

 <h2>Child Component</h2>

 <p>This is the projected content!</p>

</div>

Que. What is between @ViewChild and @ViewChildren

3. @ViewChild

- **Purpose:** Retrieves a reference to a single DOM element or directive inside the component's view.
- **Return Type:** Returns the first matching element or directive.
- **Use Case:** When you want to interact with a specific child element or directive in your component.

Import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({

 selector: 'app-example',

 template: `

 <button #myButton>Click Me</button>

 `

})

export class ExampleComponent implements AfterViewInit {

 @ViewChild('myButton') button: ElementRef; // Reference to the button element

```

ngAfterViewInit() {
  console.log(this.button.nativeElement.textContent); // Access the DOM element
  this.button.nativeElement.style.backgroundColor = 'blue'; // Change style
}
}

```

4. @ViewChildren

- **Purpose:** Retrieves references to multiple DOM elements or directives inside the component's view.
- **Return Type:** Returns a QueryList, which is a collection of the matching elements or directives.
- **Use Case:** When you want to interact with multiple child elements or directives in a component.

```

Import { Component, ViewChildren, QueryList, AfterViewInit, ElementRef } from
 '@angular/core';

```

```

@Component({
  selector: 'app-example',
  template: `
    <button #myButton>Button 1</button>
    <button #myButton>Button 2</button>
    <button #myButton>Button 3</button>
  `
})

```

```

export class ExampleComponent implements AfterViewInit {
  @ViewChildren('myButton') buttons: QueryList<ElementRef>; // Collection of all buttons

  ngAfterViewInit() {
    this.buttons.forEach((button, index) => {
      button.nativeElement.style.backgroundColor = index % 2 === 0 ? 'blue' : 'green'; // Apply
different styles
    });
  }
}

```

Que. What is the role of ng module?

The @NgModule decorator is a fundamental building block in Angular applications. It defines an Angular module and provides a way to organize an application into cohesive blocks of functionality.

It ensures modularity, scalability, and lazy loading support for better performance and maintenance

Role of @NgModule

Declares: Components, directives, and pipes in the declarations array.

Imports: Other modules (e.g., BrowserModule, FormsModule).

Exports: Makes components, directives, or pipes available to other modules.

Provides: Services in the providers array for dependency injection.

Bootstraps: Defines the root component for app initialization.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent], // Declares components, directives, pipes
  imports: [BrowserModule], // Imports other modules
  providers: [], // Provides services
  bootstrap: [AppComponent] // Defines the entry point component
})
export class AppModule {}
```

Que. What is used of angular.json

angular.json is the configuration file for Angular CLI. It defines the settings for your Angular application and workspace.

Key Uses:

- 1. Project Configuration:**
 - Specifies project details like app name, build, test, and serve configurations.
- 2. Build Settings:**
 - Define entry points, output paths, optimization settings, and file replacements for different environments.
- 3. Assets & Styles:**
 - Manage global styles (CSS/SCSS) and assets (images, fonts) to include in the app.
- 4. Scripts:**
 - Add third-party JavaScript files globally.
- 5. Environment Configurations:**
 - Set up file replacements for different environments (e.g., dev vs prod).

Que. How can we change dist or build path

To change the dist or build path in Angular:

1. Open angular.json and locate the "outputPath" property under the build section of your app configuration.
2. Update its value to your desired path (e.g., "outputPath": "custom-folder").

Que. What is pre loading

Preloading in Angular is a strategy to load lazy-loaded modules in the background after the application has been bootstrapped. This improves performance by reducing wait times when navigating to routes associated with these modules.

Example:

- **Default behavior:** Lazy-loaded modules load only when the associated route is accessed.
- **With Preloading:** Lazy-loaded modules load in advance while the app is idle.

How to enable:

Use the PreloadAllModules strategy in the RouterModule:

```
RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules });
```

This ensures better user experience without affecting the initial app load time.

Que. What is circular dependency in angular

A **circular dependency** in Angular occurs when two or more modules, components, services, or files depend on each other directly or indirectly, creating a loop. This can lead to runtime errors or unexpected behaviors.

Example:

1. **File A depends on File B, and File B depends on File A.**
2. When Angular tries to resolve these dependencies, it gets stuck in an infinite loop.

Que. Dependency injection in angular

Dependency Injection (DI) in Angular is a design pattern used to manage how components, services, and other dependencies are provided to classes. It allows objects to be injected into a class rather than the class creating the objects itself, promoting loose coupling and better testability.

Que. What is different between injecting dependency or service using inject() and constructor

| Feature | Constructor Injection | inject() Function |
|-----------------------|--|--|
| Usage Context | Used in constructors of components, services, or directives | Used outside of constructors (e.g., in <code>ngOnInit()</code> , factories, or standalone services) |
| Dependency Resolution | Angular resolves dependencies at compile-time | Dependencies are resolved at runtime using <code>inject()</code> |
| Common Use Case | Most common and preferred way of injecting dependencies | Used for dynamic or flexible dependency injection, especially in standalone components or services |
| Lifecycle | Tied to component/service lifecycle | Can be used independently of Angular lifecycle hooks (e.g., in factories or service methods) |
| When to Use | Use when you need the dependency as part of the class instance | Use when you need to inject dependencies outside the constructor, or for dynamic dependency resolution |

Note: When you use `providedIn: 'root'` in the `@Injectable` decorator, **the service is a singleton** throughout the entire Angular application.

Que. Difference between constructor and ngOnInit

1. Constructor

- **Purpose:** Initializes the class and its members when an instance is created.
- **When It Runs:** Before the Angular component lifecycle begins.
- **Use Cases:**
 - Dependency injection (injecting services into the component).
 - Basic initialization of class-level variables (not dependent on bindings or Angular's lifecycle).

2. ngOnInit

- **Purpose:** Initializes the component after Angular sets up input bindings.
- **When It Runs:** After the constructor, during the Angular component lifecycle.
- **Use Cases:**
 - Fetching data or performing tasks that rely on fully initialized input bindings.
 - Logic that depends on the DOM or Angular-specific elements being ready.

| Feature | Constructor | ngOnInit |
|----------------------|--|--|
| Purpose | Initialize class and inject dependencies | Perform Angular-specific initialization tasks |
| Lifecycle | Runs before Angular's lifecycle hooks | Runs after the constructor and bindings are set |
| Access to Input Data | Cannot access input properties | Can access input properties |
| Use Cases | Dependency injection, setting up basic class members | Fetching data, initializing logic dependent on Angular bindings or DOM |
| Execution | Always runs during component instantiation | Part of Angular's lifecycle hooks; requires <code>OnInit</code> implementation |

Que. What is AOT Compiler?

The **Ahead-of-Time (AOT) compiler** is a build-time process in Angular that compiles the application's HTML templates and TypeScript code into efficient JavaScript code before the browser downloads and executes the application. This process occurs during the build phase, rather than at runtime, which is the case with **Just-in-Time (JIT)** compilation.

Advantages of AOT:

1. **Faster Rendering:** Templates are compiled at build time, reducing the workload at runtime.
2. **Smaller Bundle Size:** Removes Angular-specific code and ships optimized JavaScript, leading to smaller bundles.
3. **Improved Security:** Detects and prevents injection attacks by compiling templates to JavaScript.
4. **Early Error Detection:** Identifies template and binding errors during the build phase, reducing runtime issues.

Que. How to get params from route

1. Define the Route: When setting up the route in your routing module, make sure the id is passed as a parameter.

```
const routes: Routes = [  
  { path: 'details/:id', component: DetailsComponent }  
];
```

2. Access the id in the Component: Use ActivatedRoute to access the route parameter in the component.

```
import { Component, OnInit } from '@angular/core';  
import { ActivatedRoute } from '@angular/router';
```

```
@Component({  
  selector: 'app-details',  
  templateUrl: './details.component.html',  
  styleUrls: ['./details.component.css']  
})  
export class DetailsComponent implements OnInit {  
  id: string | null = null;  
  
  constructor(private route: ActivatedRoute) {}  
  
  ngOnInit(): void {  
    // Access the 'id' parameter  
    this.id = this.route.snapshot.paramMap.get('id');  
  
    // Alternatively, to listen for changes if the route parameter changes dynamically:  
    // this.route.paramMap.subscribe(params => {  
    //   this.id = params.get('id');  
    // });  
  }  
}
```

Que. Tell me some Router service method

1. navigate()

- Used to navigate to a new route programmatically.
this.router.navigate(['/path', id]);

2. isActive()

- Checks whether a route is currently active.
const isActive = this.router.isActive('/path', true);

3. back()

- Navigates back to the previous route, similar to using the browser's back button.
this.router.navigate(['/previous']);

Que. What are the different attributes in angular

1. Directive Attributes

2. Binding Attributes

- **Property Binding ([])**: Binds the value of an element property to a component's property.
`<input [value]="inputValue">`
- **Event Binding (())**: Binds an event to a method in the component.
`<button (click)="handleClick()">Click me</button>`
- **Two-way Binding ([()])**: Combines property binding and event binding to create a two-way binding.
`<input [(ngModel)]="username">`

3. Input and Output Attributes

- **@Input**: Used to pass data from a parent component to a child component.
`@Input() title: string;`
- **@Output**: Used to emit events from a child component to a parent component.
`@Output() dataChanged = new EventEmitter<string>();`

4. Template Reference Variables

- Used to reference DOM elements or Angular components in templates.

`<input #inputField>`

`<button (click)="onSubmit(inputField.value)">Submit</button>`

Class and Style Binding

- **[class.className]**: Adds or removes CSS classes based on conditions.

`<div [class.active]="isActive">This div will have the 'active' class if isActive is true.</div>`

- **[style.property]**: Binds styles to an element.

`<div [style.color]="color">This div's color will be bound to the 'color' variable.</div>`

5. Angular Routing Attributes

- **[routerLink]**: Used to navigate between routes in Angular.

`<a [routerLink]="['/home']">Home</a>`

- **[routerLinkActive]**: Adds a CSS class when a link is active.

`<a [routerLink]="['/home']" routerLinkActive="active">Home</a>`

Que. What is used of environmental ts file

In Angular, the `environment.ts` file is used to store configuration settings and environment-specific variables for the application. It helps manage different environments (like development, production, staging, etc.) by separating environment-specific configurations into different files. This ensures that your app can behave differently in various environments without changing the core code.

Use of `environment.ts`:

1. **Environment-specific settings**: It holds settings like API URLs, feature flags, logging configurations, or any environment-specific logic that differs between development, production, or staging.

2. **Separation of Concerns:** By storing configurations in environment files, you avoid hardcoding these values into the application, making it easier to maintain and change configurations for different environments.

1. Development Environment (src/environments/environment.ts):

```
export const environment = {  
  production: false,  
  // staging=true for staging  
  apiUrl: 'https://dev-api.example.com',  
  logging: true  
};
```

2. Production Environment (src/environments/environment.prod.ts):

```
export const environment = {  
  production: true,  
  apiUrl: 'https://prod-api.example.com',  
  logging: false  
};
```

How It Works:

- The **environment.ts** file is used during development.
- The **environment.prod.ts** file is used when you build the app for production (ng build --prod or ng build --configuration=prod).

Note : To create environments other than **development** and **production** (e.g., **staging**):

1. **Create a new environment file** (e.g., environment.staging.ts) in the src/environments folder.
2. **Update the angular.json configuration** to add the new environment in the fileReplacements section for both build and serve configurations.

Que. What is Event Propagation

Event propagation is the process by which events propagate (or travel) through the DOM (Document Object Model) tree when triggered. It occurs in two phases:

Capturing Phase (Trickling Phase): The event starts from the root and travels down to the target element.

Bubbling Phase: After reaching the target element, the event bubbles back up to the root of the DOM.

Event Propagation Example:

```
<div id="parent" (click)="onParentClick()">  
  <button id="child" (click)="onChildClick()">Click Me</button>  
</div>
```

Que. What is Stop Event Propagation

Stopping event propagation refers to preventing the event from continuing to propagate through the DOM, either during the capturing phase or the bubbling phase. This is useful when you want to stop the event from affecting other event listeners.

stopPropagation():

- **Usage:** It prevents the event from bubbling up to parent elements or being captured by ancestors.
- **When to use:** When you want to prevent an event from triggering handlers on parent elements.

```
<div id="parent" (click)="onParentClick()">
  <button id="child" (click)="onChildClick($event)">Click Me</button>
</div>
```

```
onChildClick(event: MouseEvent) {
  event.stopPropagation(); // Stops the event from bubbling up to the parent
  console.log("Child clicked!");
}
```

```
onParentClick() {
  console.log("Parent clicked!"); // This will not be triggered after stopPropagation
}
```

Que. What is router-outlet?

`<router-outlet>` is a directive in Angular used to render components based on the active route in your application. It acts as a placeholder where the routed component will be displayed.

When you configure routes in Angular, the router-outlet tag is used in the template to specify where the matched component should be injected.

Example:

```
<!-- app.component.html -->
<div>
  <h1>Welcome to my app!</h1>
  <router-outlet></router-outlet> <!-- Where routed components will appear -->
</div>
```

Explanation:

1. **Routing Setup:** You configure routes in your app, specifying which components should be displayed for specific paths.
2. **Rendering Components:** When the user navigates to a specific route, Angular will load the corresponding component inside the `<router-outlet>` in the template.

Example of Routing Configuration:

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';
```

```
const routes: Routes = [
  { path: 'home', component: HomeComponent },
```

```
{ path: 'about', component: AboutComponent },  
{ path: '', redirectTo: '/home', pathMatch: 'full' }  
];
```

```
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})  
export class AppRoutingModule {}
```

How it works:

- **Home route** (/home) will display the HomeComponent in the router-outlet.
- **About route** (/about) will display the AboutComponent in the router-outlet.

Summary:

<router-outlet> is a placeholder directive that displays routed components based on the current active route in your Angular application.

Que. How to Create Multiple router-outlet

In Angular, we can have multiple router-outlet directives in your application to handle **nested routing** or **multiple route views**. Each router-outlet can be linked to a specific part of your application where different views/components are rendered.

How to Use Multiple router-outlet:

1. **Basic Example with Multiple router-outlet:** You can define multiple router-outlet elements in your template. Each one will correspond to a different route view.
2. **Example Scenario:**
 - **Primary Outlet:** Main view.
 - **Secondary Outlet:** Sidebar or a modal.

Example:

1. **app.component.html** (Main Template):

```
<div class="main-content">  
  <h1>Welcome to the App</h1>  
  <router-outlet></router-outlet> <!-- Primary outlet -->  
</div>
```

```
<div class="sidebar">  
  <router-outlet name="sidebar"></router-outlet> <!-- Secondary outlet -->  
</div>
```

2. **app-routing.module.ts** (Routing Configuration):

```
import { NgModule } from '@angular/core';  
import { RouterModule, Routes } from '@angular/router';  
import { HomeComponent } from './home/home.component';  
import { AboutComponent } from './about/about.component';  
import { SidebarComponent } from './sidebar/sidebar.component';
```

```
const routes: Routes = [
  {
    path: 'home',
    component: HomeComponent,
  },
  {
    path: 'about',
    component: AboutComponent,
  },
  {
    path: 'sidebar',
    component: SidebarComponent,
    outlet: 'sidebar' // Named outlet for sidebar
  }
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

How It Works:

- **Main router-outlet:** This renders components for routes like /home or /about.
- **Named router-outlet** (<router-outlet name="sidebar">): This outlet is specifically for rendering routes that are directed to the named outlet (sidebar in this case).

Navigating to a Named Outlet:

To navigate to a route targeting a specific outlet, you can specify the outlet in the route configuration using the outlets property.

For example, to navigate to the sidebar route:

```
this.router.navigate([{ outlets: { sidebar: ['sidebar'] } }]);
```

This will load the SidebarComponent inside the <router-outlet name="sidebar"> in the template.

Summary:

- You can use multiple router-outlet directives for different views in your app.
- You specify which outlet to use by giving it a name.
- For routing, you can direct routes to a specific outlet by configuring them with the outlet property.

Que. What is Tree Shaking?

Tree Shaking is a process in JavaScript (and in frameworks like Angular) where unused code is eliminated during the build process. It helps reduce the size of the final bundle by removing code that is not actually used in the application.

How Does It Work?

Tree shaking works by analyzing the **import/export** statements and dependencies in your code. During the build process, unused exports or functions are marked and eliminated from the final output bundle.

How to Enable Tree Shaking in Angular:

Angular uses **Webpack** under the hood, which has built-in support for tree shaking when you build the project with **--prod** flag.
ng build --prod

Benefits of Tree Shaking:

1. **Smaller Bundles:** Reduces the final bundle size, improving loading times.
2. **Faster Performance:** With less code to execute, the application runs faster.
3. **Efficient Development:** Eliminates dead or unused code automatically, so developers don't have to manually optimize dependencies.

Que. I want to fetch data before a component is called

To fetch data before a component is called in Angular, you can use **Resolvers**. A **resolver** is a service that resolves the data before the route is activated and before the component is loaded. Here's how you can implement it:

Steps to Use a Resolver:

1. **Create a Resolver Service:**
 - The resolver fetches the required data before the route is activated.

```
import { Injectable } from '@angular/core';
import { Resolve } from '@angular/router';
import { Observable } from 'rxjs';
import { MyDataService } from './my-data.service';
```

```
@Injectable({
  providedIn: 'root'
})
export class DataResolver implements Resolve<any> {

  constructor(private dataService: MyDataService) {}

  resolve(): Observable<any> {
    return this.dataService.fetchData(); // This fetches data from the API or service
  }
}
```

2. **Define the Resolver in Routes:**

- Use the resolve property in your routing module to specify the resolver.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { MyComponent } from './my.component';
```

```
import { DataResolver } from './data.resolver';

const routes: Routes = [
  {
    path: 'my-path',
    component: MyComponent,
    resolve: {
      data: DataResolver // The data key is used to store the resolved data
    }
  }
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

3. Access the Resolved Data in the Component:

- In the component, you can access the resolved data using `ActivatedRoute`.

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
```

```
@Component({
  selector: 'app-my',
  templateUrl: './my.component.html',
  styleUrls: ['./my.component.css']
})
export class MyComponent implements OnInit {
  data: any;
```

```
  constructor(private route: ActivatedRoute) {}
```

```
  ngOnInit(): void {
    // Get the resolved data
    this.data = this.route.snapshot.data['data'];
  }
}
```

Summary:

- **Resolvers** fetch data before the component is activated.
- You specify the resolver in your routing configuration.
- The data is accessed in the component via `ActivatedRoute`.

This ensures that the necessary data is fetched and available before the component loads.

Que. What is Angular Resolver

A **resolver** in Angular is a service that pre-fetches data before a route is activated, ensuring that the required data is available before the component is loaded. It is used to resolve data asynchronously, making sure the component only loads after the necessary data has been fetched.

Key Points:

1. **Pre-fetch Data:** It fetches data before the component is instantiated.
2. **Used in Routing:** Defined in the route configuration, the resolve property specifies which resolver to use.
3. **Ensures Data Availability:** Guarantees that the component receives all required data before being displayed.

Que. Give me Real-Time Scenario:

Using a Resolver in an E-commerce Application

Scenario: Product Details Page

Imagine you're building an e-commerce platform where users can view detailed information about products. Each product has an associated page that shows detailed information like product name, description, price, and customer reviews.

Problem:

When a user navigates to a product's detail page, you want to ensure that all product data (such as details and reviews) is fetched from the server **before** the component is displayed. If the data is not ready when the page loads, the user might see an empty or loading state, leading to a poor user experience.

Solution: Use a Resolver

You can use an **Angular Resolver** to fetch the product data **before** the component is activated, ensuring that the product details are available when the component is loaded.

Que. How to update the cart count in the HeaderComponent when an item is added to the cart from the ProductComponent ?

Steps:

1. **Create a Shared Cart Service** with BehaviorSubject with initial value zero to store and manage the cart count.
2. **Inject the Service** into your ProductComponent and HeaderComponent to share and update the cart count.
3. **Update the cart count** in the AddToCartComponent when an item is added to the cart.

Que. Difference Between Local Storage and Session Storage

Persistence Duration:

- **Local Storage:** Data persists even when the browser is closed and reopened. It remains until explicitly deleted by the user or through code.
- **Session Storage:** Data is cleared when the browser tab is closed. It only persists for the duration of the page session (i.e., while the tab or browser window is open).

Scope:

- **Local Storage:** Data is accessible across different browser tabs and windows, as long as they are from the same origin (same domain).
- **Session Storage:** Data is only accessible in the same tab or window. Each tab has its own separate storage space.

Storage Size:

- Both **Local Storage** and **Session Storage** have a storage limit, typically around **5MB** per domain, but this may vary slightly across different browsers.

Use Case:

- **Local Storage:** Useful for storing data that should persist across sessions (e.g., user preferences, themes, authentication tokens).
- **Session Storage:** Useful for storing temporary data that should only exist for a single page session (e.g., form data during a single session).

Example:

- **Local Storage:**

```
// Set item in localStorage
```

```
localStorage.setItem('username', 'JohnDoe');
```

```
// Get item from localStorage
```

```
let username = localStorage.getItem('username');
```

```
console.log(username); // Output: JohnDoe
```

- **Session Storage:**

```
// Set item in sessionStorage
```

```
sessionStorage.setItem('sessionToken', 'abc123');
```

```
// Get item from sessionStorage
```

```
let sessionToken = sessionStorage.getItem('sessionToken');
```

```
console.log(sessionToken); // Output: abc123
```

Que. Encapsulation in Angular (

In Angular, **encapsulation** is achieved using **View Encapsulation**, which controls how styles are applied to components. There are four types:

1. **Emulated (Default):** Styles are scoped to the component, using unique attributes to prevent global styles from affecting it.
 - **Example:** Styles only apply within the component's template.
2. **None:** Styles are global and affect the entire application.
 - **Example:** encapsulation: ViewEncapsulation.None
3. **Shadow DOM:** Styles are completely isolated to the component, using the browser's native Shadow DOM.
 - **Example:** encapsulation: ViewEncapsulation.ShadowDom
4. **::ng-deep (Deprecated):** Allows styles to affect child components, bypassing encapsulation.
 - **Example:** ::ng-deep app-child { color: red; }

Example:

```
@Component({ selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css'],  
  encapsulation: ViewEncapsulation.ShadowDom })
```

Summary:

Angular's **CSS encapsulation** helps in scoping styles to a component to avoid unwanted global style leaks. By default, Angular uses **Emulated** encapsulation.

Que. What is forRoot() and forChild() ?

Both forRoot() and forChild() are used to configure routes in Angular, but they serve different purposes depending on where they are used.

1. forRoot()

- **Purpose:**

- forRoot() is used to configure the root routing for the application. It should be called only once in the application (usually in the **root module**, typically AppModule).
- It defines the **main routing** configuration for the application.

- **When to Use:**

- Call forRoot() when you're setting up **global routes** that are needed by the entire application. It's used for the **main application module**.

- **Example:**

```
import { NgModule } from '@angular/core';  
import { RouterModule, Routes } from '@angular/router';  
import { HomeComponent } from './home/home.component';
```

```
const routes: Routes = [  
  { path: '', component: HomeComponent }  
];
```

```
@NgModule({  
  imports: [RouterModule.forRoot(routes)], // Calling forRoot() here  
  exports: [RouterModule]  
})  
export class AppRoutingModule { }
```

2. forChild()

- **Purpose:**

- forChild() is used to define **child routes** or **routes for feature modules**.
- It is typically used when setting up **feature module routing** or **lazy-loaded modules**. Unlike forRoot(), forChild() does not define top-level routes; instead, it defines routes that are nested under the root or parent routes.

- **When to Use:**

- Use `forChild()` in **feature modules** or **child routes** when you need to add routes inside specific modules (e.g., when you have a lazy-loaded feature module).

- **Example:**

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { FeatureComponent } from './feature/feature.component';
```

```
const featureRoutes: Routes = [
  { path: 'feature', component: FeatureComponent }
];
```

```
@NgModule({
  imports: [RouterModule.forChild(featureRoutes)], // Calling forChild() here
  exports: [RouterModule]
})
export class FeatureRoutingModule { }
```

Summary:

- **forRoot()** is used for **global routing configuration** in the root module, and it should only be called once in the app.
- **forChild()** is used for **feature module or child routes** in child modules and can be used multiple times.

Que. What is used of Constructor in Angular

In Angular, the **constructor** is a special method within a class that is used for **initializing the class** and **injecting dependencies**

RXJS

What is an Observable?

An **Observable** is a data producer that emits values asynchronously over time. It represents a **stream of data** that can be observed and subscribed by subscriber.

Core Concepts of Observables:

1. Emit Values: An Observable emits a sequence of values (of any type) over time.
2. Subscribe: You "subscribe" to an Observable to listen for these emitted values.
3. Observable Operators: RxJS provides many operators to transform, filter, or combine observables.

How Does an Observable Work?

An Observable works on the Observer Pattern, where:

- Observable: The source of the data (produces values).
- Observer: The consumer of the data (receives the values).
- Subscription: A way to connect the Observable to the Observer.

Que. Types of RxJs Operator?

1. Creation Operators

Used to create new Observables.

- of: Create an Observable from a sequence of values.
- from: Create an Observable from an array, Promise, or iterable.
- fromEvent: Create an Observable from DOM events.
- interval: Emit values at a fixed time interval.
- timer: Create an Observable that emits after a specified delay or at intervals.
- range: Emit a sequence of numbers within a range.
- throwError: Create an Observable that emits an error.

2. Transformation Operators

Used to transform the emitted values.

- map: Apply a function to each value emitted by the source Observable.
- mapTo: Map every value emitted to the same constant value.
- switchMap: Switch to a new inner Observable, canceling the previous one.
- mergeMap (or flatMap): Map each value to an inner Observable and merge their outputs.
- concatMap: Map each value to an inner Observable and subscribe to them sequentially.
- exhaustMap: Map to inner Observable, ignoring new values until the inner Observable completes.
- scan: Accumulate values and emit the accumulated result.
- buffer: Collect values emitted by the source Observable and emit them as an array.

3. Filtering Operators

Used to filter or limit the emitted values.

- filter: Emit only those values that pass a condition.

- take: Take only the first n values and then complete.
- takeWhile: Take values while a condition is true.
- takeUntil: Take values until another Observable emits.
- skip: Skip the first n values.
- skipWhile: Skip values while a condition is true.
- distinct: Emit values only if they are distinct from the previous one.
- distinctUntilChanged: Emit values only if they change from the previous one.
- first: Take only the first value.
- last: Take only the last value.

4. Combination Operators

Used to combine multiple Observables.

- merge: Combine multiple Observables into one by merging their emissions.
- concat: Combine multiple Observables, subscribing to each sequentially.
- combineLatest: Emit values from multiple Observables when any emits, combining the latest values.
- forkJoin: Wait for all Observables to complete, then emit their last values as an array.
- zip: Combine multiple Observables by emitting their values as arrays when all emit.
- startWith: Emit a specified value before the source Observable emits anything.

5. Error Handling Operators

Used to handle errors in Observables.

- catchError: Handle errors and return a new Observable.
- retry: Retry the source Observable a specified number of times on error.
- retryWhen: Retry the Observable based on a custom logic.
- onErrorResumeNext: Continue to the next Observable in case of an error.

6. Utility Operators

Used for debugging or performing side effects.

- tap (formerly do): Perform a side effect for each value emitted.
- delay: Delay emissions by a specified time.
- timeout: Throw an error if the Observable doesn't emit within a specified time.
- finalize: Perform a side effect when the Observable completes or errors.
- delayWhen: Delay emissions based on another Observable.
- toPromise: Convert an Observable to a Promise.

7. Multicasting Operators

Used for sharing a single Observable among multiple subscribers.

- share: Share a single subscription among multiple subscribers.
- shareReplay: Share and replay the latest emitted values to new subscribers.
- publish: Turn a cold Observable into a hot Observable.
- multicast: Share an Observable using a Subject.

8. Conditional and Boolean Operators

Used for conditional logic.

- `defaultIfEmpty`: Emit a default value if the source Observable is empty.
- `every`: Emit true if all values pass a condition.
- `find`: Emit the first value that matches a condition.
- `findIndex`: Emit the index of the first value that matches a condition.

9. Higher-Order Mapping Operators

Used for managing multiple Observables.

- `concatAll`: Flatten an Observable-of-Observables sequentially.
- `mergeAll`: Flatten an Observable-of-Observables concurrently.
- `switchAll`: Flatten an Observable-of-Observables, canceling previous inner Observables.
- `exhaust`: Subscribe to an Observable-of-Observables, ignoring new ones until the current completes.

10. Subject Operators

Subjects are a special kind of Observable.

- `Subject`: Acts as both Observable and Observer.
- `BehaviorSubject`: Emits the last emitted value to new subscribers.
- `ReplaySubject`: Replays all or specified emissions to new subscribers.
- `AsyncSubject`: Emits only the last value upon completion.

1. map Operator

What It Does:

`map` transforms each value emitted by an observable by applying a function to it. It is often used to modify or reshape the data before using it in the application.

Real-Time Scenario:

You have an API that returns a list of users. Before displaying these user names on the UI, you want to convert them to uppercase.

```
// API response: [{ name: 'John' }, { name: 'Doe' }]
this.http.get<User[]>('api/users').pipe(
  map(users => users.map(user => user.name.toUpperCase())) // Transform to uppercase
).subscribe(names => console.log(names)); // Output: ['JOHN', 'DOE']
```

2. filter Operator

What It Does:

`filter` allows you to emit only the values that meet a certain condition, ignoring the rest. It's useful for narrowing down results.

Real-Time Scenario:

You want to display only active users in the UI, filtering out inactive ones.

```
// API response: [{ name: 'John', active: true }, { name: 'Doe', active: false }]
this.http.get<User[]>('api/users').pipe(
  map(users => users.filter(user => user.active)) // Filter only active users
).subscribe(activeUsers => console.log(activeUsers));
// Output: [{ name: 'John', active: true }]
```

3. switchMap Operator

What It Does:

switchMap maps a value to a new observable, **canceled any previous observable** if a new value arrives before the previous one completes.

Real-Time Scenario:

You're building a search bar with autocomplete functionality. Each keystroke triggers an API call, but only the latest search result should be displayed.

```
import { fromEvent } from 'rxjs';
import { map, switchMap } from 'rxjs/operators';

// Example: HTML <input #searchInput type="text" />
fromEvent(this.searchInput.nativeElement, 'input').pipe(
  map(event => (event.target as HTMLInputElement).value), // Get the input value
  switchMap(query => this.http.get(`api/search?q=${query}`)) // Fetch search results
).subscribe(results => console.log(results));
```

4. mergeMap Operator

What It Does:

mergeMap maps each value to an observable, **allowing multiple inner subscriptions to run concurrently**. Unlike switchMap, it does not cancel previous subscriptions.

Real-Time Scenario:

You want to fetch a user's details and, after that, fetch their orders without waiting for the first request to complete before starting the next.

Example:

```
this.http.get<User>('api/user/1').pipe(
  mergeMap(user => this.http.get<Order[]>(`api/orders/${user.id}`)) // Fetch orders after user
).subscribe(orders => console.log(orders));
```

5. concatMap

What It Does:

concatMap maps each value to an observable and subscribes to them **one at a time, in order**. It waits for the current observable to complete before moving to the next.

Real-Time Scenario:

You want to process multiple form submissions sequentially (one after another), ensuring the previous one finishes before starting the next.

Example:

```
const formData = [data1, data2, data3]; // Form submission data
```

```
from(formData).pipe(
  concatMap(data => this.http.post('api/submit', data)) // Sequentially process each
  submission
).subscribe(response => console.log(response));
```

Explanation:

1. from(formData): Emits each form data one at a time.
2. concatMap(data => this.http.post(...)): Sends form data sequentially.
3. Ensures each HTTP request completes before the next one starts.

6. exhaustMap**What It Does:**

exhaustMap ignores new emissions while a previous observable is still active. It's useful for preventing duplicate or overlapping requests.

Real-Time Scenario:

You have a "Login" button, and you want to ignore duplicate login clicks while the API call is in progress.

Example:

```
fromEvent(this.loginButton.nativeElement, 'click').pipe(
  exhaustMap(() => this.http.post('api/login', this.loginForm.value)) // Ignore duplicate clicks
).subscribe(response => console.log(response));
```

Explanation:

1. fromEvent(...): Listens for login button clicks.
2. exhaustMap(...): Starts the login API call but ignores new clicks until the current request completes.
3. Prevents duplicate login requests.

3. catchError**What It Does:**

catchError allows you to handle errors in the observable stream and optionally return a fallback value or throw a new error.

Real-Time Scenario:

You want to display an error message if an API call fails, without breaking the observable chain.

Example:

```
this.http.get<User[]>('api/users').pipe(
  catchError(error => {
    console.error('Error fetching users:', error);
    return of([]); // Return an empty array as a fallback
  })
).subscribe(users => console.log(users));
```

Explanation:

1. `this.http.get(...)`: Fetches users from the API.
2. `catchError(error => ...)`: Logs the error and returns an empty array to prevent breaking the chain.
3. Subscribed result logs an empty array if an error occurs.

4. combineLatest**What It Does:**

`combineLatest` combines the latest values from multiple observables whenever one of them emits.

Real-Time Scenario:

You have two dropdowns, and you want to fetch data based on the selected values of both.

Example:

```
combineLatest([
  this.dropdown1.valueChanges,
  this.dropdown2.valueChanges
]).pipe(
  switchMap(([value1, value2]) => this.http.get(`api/data?val1=${value1}&val2=${value2}`))
).subscribe(data => console.log(data));
```

Explanation:

1. `combineLatest(...)`: Listens for changes in both dropdowns.
2. `switchMap(...)`: Fetches data based on the latest values of both dropdowns.
3. Subscribed result logs data fetched using both values.

5. forkJoin**What It Does:**

`forkJoin` waits for all observables to complete and emits their final values as an array.

Real-Time Scenario:

You want to fetch user details, orders, and settings in parallel and process the data only after all requests are complete.

Example:

```
forkJoin({
  user: this.http.get<User>('api/user/1'),
  orders: this.http.get<Order[]>('api/orders/1'),
  settings: this.http.get<Settings>('api/settings/1')
}).subscribe(result => {
  console.log(result.user, result.orders, result.settings);
});
```

Explanation:

1. `forkJoin(...)`: Runs all HTTP requests in parallel.
2. Emits the final results once all requests are complete.
3. Subscribed result provides all data (user, orders, settings) at once.

6. debounceTime

What It Does:

debounceTime delays the emission of values by a specified time, ignoring intermediate values. It's useful for handling rapid events like user input.

Real-Time Scenario:

You're building a search bar, and you want to wait for the user to stop typing before sending the search query.

Example:

```
fromEvent(this.searchInput.nativeElement, 'input').pipe(
  debounceTime(300), // Wait 300ms after the user stops typing
  map(event => (event.target as HTMLInputElement).value),
  switchMap(query => this.http.get(`api/search?q=${query}`))
).subscribe(results => console.log(results));
```

Explanation:

1. debounceTime(300): Ignores input events until 300ms of inactivity.
2. Prevents sending unnecessary API requests for each keystroke.

7. retry

What It Does:

retry re-subscribes to an observable a specified number of times after an error.

Real-Time Scenario:

You're making an API call that may fail due to network issues, and you want to retry it a few times before giving up.

Example:

```
this.http.get<User[]>('api/users').pipe(
  retry(3), // Retry up to 3 times if an error occurs
  catchError(error => {
    console.error('API call failed after 3 retries:', error);
    return of([]);
  })
).subscribe(users => console.log(users));
```

Explanation:

1. retry(3): Retries the HTTP request up to 3 times.
2. Prevents giving up on transient errors like network issues.

8. takeUntil

What It Does:

The takeUntil operator listens to a **notifier observable**. It emits values from the source observable until the notifier emits a value or completes. Once the notifier emits, the subscription automatically stops.

Real-Time Scenario:

You want to listen to live updates (e.g., real-time user location) **only while the component is active** and stop the updates when the component is destroyed.

Example:

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.locationService.getLiveLocation().pipe(
    takeUntil(this.destroy$) // Stop listening when the component is destroyed
  ).subscribe(location => {
    console.log('Live Location:', location);
  });
}

ngOnDestroy() {
  this.destroy$.next(); // Emit a value to complete the notifier
  this.destroy$.complete();
}
```

Explanation:

1. **When to Use:** Use takeUntil to clean up subscriptions automatically when the component is destroyed or some condition is met.
2. **Notifier Observable:** The destroy\$ subject acts as a notifier. Once it emits a value in ngOnDestroy, the subscription ends.

Difference Between `mergeMap` and Other Operators

Operator	Cancels Previous?	Concurrent Execution?	Use Case
<code>mergeMap</code>	✗ No	✓ Yes	Concurrent API calls, file uploads, or handling multiple streams.
<code>switchMap</code>	✓ Yes	✗ No	Debouncing, switching between streams, or dependent requests.
<code>concatMap</code>	✗ No	✗ No	Sequential processing of tasks or API calls (one at a time).
<code>exhaustMap</code>	✓ Ignores	✗ No	Avoiding duplicate requests (ignores new ones while a task is in flight).

Que. difference between setTimeout() and setInterval()

The primary difference between setTimeout() and setInterval() lies in how they are used and how often the callback function is executed:

1. setTimeout()

- **Purpose:** Executes a function **once** after a specified delay.
- **Syntax:** setTimeout(callback, delay)

```
// Executes after 2 seconds (2000 milliseconds)
setTimeout(() => {
  console.log('This will run after 2 seconds!');
}, 2000);
```

2. setInterval()

- **Purpose:** Executes a function **repeatedly**, with a fixed time interval between each execution.
- **Syntax:** setInterval(callback, interval)
- **How it works:** It runs the callback function **repeatedly** every specified interval (in milliseconds), until it is cleared.
- **Use case:** For recurring tasks, like updating a clock or sending requests at regular intervals.

```
// Executes every 2 seconds (2000 milliseconds)
setInterval(() => {
  console.log('This will run every 2 seconds!');
}, 2000);
```

Que. Explain about merge() and concat();

1. merge() - Real-World Example: Handling Multiple API Calls Concurrently

Scenario: You have a dashboard application that fetches data from several sources (e.g., user profile, notifications, and recent activities). You want to display all the data concurrently, as soon as any of the API calls return a result. You don't care about the order in which the data is fetched, as long as it's displayed when ready.

Use Case:

- **Merge** is ideal when you don't care about the order of emissions and want to fetch and display data from multiple API endpoints simultaneously.

Example:

```
import { of, merge } from 'rxjs';
import { delay } from 'rxjs/operators';
```

```
// Simulating API calls
```

```
const userProfileApi = of('User Profile Data').pipe(delay(1000)); // 1 second delay
const notificationsApi = of('Notifications Data').pipe(delay(2000)); // 2 seconds delay
const activitiesApi = of('Recent Activities').pipe(delay(3000)); // 3 seconds delay
```

```
// Merging the observables to fetch data concurrently
```

```
merge(userProfileApi, notificationsApi, activitiesApi).subscribe(data => {
```

```
console.log('Received:', data);
});
```

Output:

Received: User Profile Data // After 1 second
Received: Notifications Data // After 2 seconds
Received: Recent Activities // After 3 seconds

- **How it works:** All three API calls are made concurrently. The first one to return data (User Profile Data after 1 second) gets emitted, followed by the next ones in the order they resolve. The values are emitted as soon as they are available, without waiting for others to finish.

2. concat() - Real-World Example: Sequential Task Execution

Scenario: You have a multi-step form where each step depends on the previous one. For example:

1. **Step 1:** User selects a product.
2. **Step 2:** User provides personal details.
3. **Step 3:** User confirms the order.

You want each step to complete before moving to the next. For example, you shouldn't send the order for confirmation until the user's details are successfully saved.

Use Case:

- **Concat** is ideal for scenarios where tasks need to be executed **sequentially**. You want to make sure one task completes before the next begins.

Example:

typescript

CopyEdit

```
import { of, concat } from 'rxjs';
import { delay } from 'rxjs/operators';
```

```
// Simulating sequential tasks (API calls or operations)
```

```
const selectProduct = of('Product Selected').pipe(delay(1000)); // 1 second delay
const saveUserDetails = of('User Details Saved').pipe(delay(2000)); // 2 seconds delay
const confirmOrder = of('Order Confirmed').pipe(delay(1000)); // 1 second delay
```

```
// Concatenating tasks to ensure they happen sequentially
```

```
concat(selectProduct, saveUserDetails, confirmOrder).subscribe(data => {
  console.log(data);
});
```

Output:

mathematica

CopyEdit

Product Selected // After 1 second
User Details Saved // After 3 seconds
Order Confirmed // After 4 seconds

- **How it works:** `concat()` ensures that each step happens only after the previous one completes. So, `saveUserDetails` will wait for `selectProduct` to finish, and `confirmOrder` will wait for `saveUserDetails` to finish. The tasks are completed in sequence.

Que. Explain Subject, BehaviorSubject, ReplaySubject, AsyncSubject

In Angular, Subject is commonly used in services for communication between components.

1. Subject

- **Use case:** When you need to broadcast values to multiple subscribers without keeping track of the last emitted value.

Real-World Example:

Think of a **chatroom** where multiple users can join and receive messages. A Subject can be used to broadcast messages to all the users.

Scenario:

A chatroom where multiple users join, and the chat messages are broadcasted to all subscribers (users).

typescript

CopyEdit

```
import { Subject } from 'rxjs';
```

```
// Create a Subject to broadcast messages
```

```
const chatSubject = new Subject();
```

```
// Subscriber 1 - A user joins the chat and starts receiving messages
```

```
chatSubject.subscribe(message => {
  console.log('User 1:', message);
});
```

```
// Subscriber 2 - Another user joins and starts receiving messages
```

```
chatSubject.subscribe(message => {
  console.log('User 2:', message);
});
```

```
// Broadcasting messages
```

```
chatSubject.next('Hello, User 1!');
```

```
chatSubject.next('Hi, User 2!');
```

```
chatSubject.next('New Message!');
```

- **How it works:**
 - The Subject sends messages to all active subscribers when `next()` is called.
 - New subscribers **don't receive previous messages**—they only get messages emitted after they subscribe.

2. BehaviorSubject

- **Use case:** When you need to always emit the **latest value** to new subscribers (i.e., it remembers the last emitted value).

Real-World Example:

Imagine you have a **user profile** service in your application, and you want to show the most recent user data (like user name, profile picture, etc.) to any new user who subscribes.

Scenario:

A profile service that holds the current user's information and always gives the latest info to any new subscribers.

```
import { BehaviorSubject } from 'rxjs';
```

```
// Create a BehaviorSubject with an initial value
```

```
const userProfile = new BehaviorSubject({ name: 'John', age: 30 });
```

```
// Subscriber 1 - A user views the profile
```

```
userProfile.subscribe(user => {  
  console.log('Subscriber 1:', user); // Output: { name: 'John', age: 30 }  
});
```

```
// Update the user profile
```

```
userProfile.next({ name: 'Jane', age: 25 });
```

```
// Subscriber 2 - A new user views the profile, receives the latest value immediately
```

```
userProfile.subscribe(user => {  
  console.log('Subscriber 2:', user); // Output: { name: 'Jane', age: 25 }  
});
```

- **How it works:**
 - BehaviorSubject always **remembers the last value**.
 - When a new subscriber subscribes, they **immediately receive the most recent value**.

3. ReplaySubject

- **Use case:** When you want to emit multiple previous values to new subscribers. You can specify how many past values it should cache and replay.

Real-World Example:

Think of a **news feed** or a **stock price update service**. A ReplaySubject will allow you to replay the last few updates when a new subscriber joins.

Scenario:

A service that broadcasts news updates to all subscribers and replays the last 3 updates when a new user joins.

typescript

CopyEdit

```
import { ReplaySubject } from 'rxjs';
```

```
// Create a ReplaySubject with a buffer size of 3
const newsFeed = new ReplaySubject(3);

// Emitting some news updates
newsFeed.next('News Update 1');
newsFeed.next('News Update 2');
newsFeed.next('News Update 3');
newsFeed.next('News Update 4');

// Subscriber 1 - Gets the last 3 news updates when they join
newsFeed.subscribe(news => {
  console.log('Subscriber 1:', news); // Output: News Update 2, News Update 3, News Update 4
});

// Subscriber 2 - New user also gets the last 3 updates
newsFeed.subscribe(news => {
  console.log('Subscriber 2:', news); // Output: News Update 2, News Update 3, News Update 4
});
```

- **How it works:**
 - ReplaySubject stores a **buffer of recent values** (in this case, 3).
 - New subscribers **immediately receive the last 3 values** when they subscribe, even if those values were emitted earlier.

4. AsyncSubject

- **Use case:** When you want to emit only the **final value** upon completion of the observable (emits only the last value after complete()).

Real-World Example:

Imagine an **order tracking system** where you only care about the **final status** of an order (e.g., "Delivered") once the entire order process is complete. You don't want to receive updates until the final one is available.

Scenario:

An order processing service where a customer gets the final status of their order after the order processing is complete.

typescript

CopyEdit

```
import { AsyncSubject } from 'rxjs';
```

```
// Create an AsyncSubject
const orderStatus = new AsyncSubject();
```

```
// Emitting intermediate statuses
orderStatus.next('Order Placed');
```

```
orderStatus.next('Order Dispatched');
orderStatus.next('Order Shipped');
```

```
// Complete the process with the final status
orderStatus.next('Order Delivered');
orderStatus.complete();
```

```
// Subscriber - Receives only the final value once the observable completes
orderStatus.subscribe(status => {
  console.log('Final Status:', status); // Output: Final Status: Order Delivered
});
```

- **How it works:**

- AsyncSubject will only **emit the last value** after the observable calls complete().
- No subscriber will receive any value until complete() is invoked, at which point the last emitted value ('Order Delivered') is broadcasted.

Comparison in Summary:

Subject Type	Use Case	Emits	Subscribers' Behavior
Subject	Broadcasting messages to multiple subscribers	Sends all emitted values to subscribers.	New subscribers don't receive previous values.
BehaviorSubject	Emitting the most recent value to new subscribers	Remembers and emits the last emitted value to new subscribers.	New subscribers immediately get the last emitted value.
ReplaySubject	Replaying a specified number of previous values to new subscribers	Remembers a buffer of past emitted values and replays them.	New subscribers get the buffered past values.
AsyncSubject	Emitting only the final value after completion	Emits the last value only when the observable completes.	New subscribers only get the final value after completion.

Que. What is different between promise and observable

OR difference between Promise/fetch API and Observable/subscribe

1. Multiple vs Single Values:

- **Promise:** Handles **single asynchronous value**. Once the Promise resolves or rejects, it will emit a value or an error and then stop. It can't emit multiple values.
 - **Example:** A HTTP request that resolves with data or fails with an error.
- **Observable:** Can handle **multiple asynchronous values** over time. An Observable can emit multiple values before completing or erroring out.
 - **Example:** Streaming data, user input events, or continuous sensor readings.

2. Eager vs Lazy:

- **Promise: Eager.** A Promise starts executing as soon as it is created, regardless of whether anyone is subscribed to it.
- **Observable: Lazy.** An Observable only starts emitting values when it is **subscribed**. If you never subscribe to it, it won't execute.

3. Cancellation:

- **Promise:** Once a Promise is initiated, it **can't be cancelled**.
- **Observable:** You can **unsubscribe** to an Observable at any point, which stops it from emitting further values. This provides better control over resource management.

4. Operators & Transformation:

- **Promise:** Promises provide basic chaining with `.then()`, `.catch()`, and `.finally()` for handling resolved values, errors, and cleanup. Limited to basic transformations.
- **Observable:** Observables have a rich set of operators (like `map`, `filter`, `mergeMap`, `switchMap`, etc.) that can be used to transform, filter, and combine streams of data. This makes Observables much more powerful for handling complex asynchronous scenarios.

5. Error Handling:

- **Promise:** If a Promise is rejected, it goes to the nearest `.catch()` handler. Once it resolves or rejects, the value is final, and it can't be re-emitted.
- **Observable:** Observables allow multiple emissions, including errors. When an error occurs, it can be caught with `.catchError()`, and it doesn't complete the Observable. You can handle errors as a stream itself.

6. Memory Management:

- **Promise:** Since promises can't be cancelled, they could result in memory issues if not properly handled.
- **Observable:** You can unsubscribe from an Observable to clean up memory, which is useful for long-running operations (like HTTP requests, WebSocket, event listeners).

When to use Promise vs Observable:

- **Use Promises** when you need to handle a **single asynchronous event** (e.g., HTTP request, file reading, or one-time computations).
- **Use Observables** when you need to handle **multiple asynchronous events** or streams of data (e.g., WebSockets, user events, or continuous data flow).

Que. Difference between Observable and Subject ?

1. **Observable** is a unicast data stream, emitting values to each subscriber individually when they subscribe.
2. **Subject** is a multicast data stream, sharing the same values to all subscribers and can emit values manually using `next()`.
3. **Observable** doesn't hold state, while **Subject** can retain and share the latest value with new subscribers (e.g., `BehaviorSubject`).

Que. What is ChangeDetectorRef in Angular?

ChangeDetectorRef is a class in Angular that provides control over **change detection** for a component. It allows us to **manually trigger, detach, or reattach** the change detection mechanism.

Key Methods:

1. **detectChanges()** – Manually triggers change detection.

```
this.cdr.detectChanges();
```

2. **detach()** – Stops automatic change detection.

```
this.cdr.detach();
```

3. **reattach()** – Resumes automatic change detection.

```
this.cdr.reattach();
```

Que. What is NgZone in Angular?

NgZone is a service in Angular that helps manage and control **change detection**. By default, Angular runs change detection for every asynchronous operation (like setTimeout, setInterval, or HTTP requests), which can impact performance.

With NgZone, you can execute code **outside Angular's change detection cycle**, reducing unnecessary UI updates and improving performance.

📌 When to Use NgZone?

Use NgZone when performing **background tasks** like:

- ✓ Running **timers** (setTimeout, setInterval).
- ✓ Handling **WebSocket updates**.
- ✓ Performing **heavy computations**.

◆ Example: Running Code Outside Angular's Change Detection

By default, Angular triggers change detection when setTimeout completes. But using NgZone.runOutsideAngular(), we can **prevent unnecessary change detection**.

📄 Without NgZone (Change Detection Runs Automatically)

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-example',
  template: ` <button (click)="startTimer()">Start Timer</button> `
})
export class ExampleComponent {
  counter = 0;

  startTimer() {
    setInterval(() => {
      this.counter++;
      console.log(` Counter: ${this.counter}`);
    }, 1000);
  }
}
```

● **Problem:** Every second, Angular **re-renders the entire component** because `setInterval` modifies `counter`.

With NgZone (Optimized Performance)

```
import { Component, NgZone } from '@angular/core';
```

```
@Component({
  selector: 'app-example',
  template: `<button (click)="startTimer()">Start Timer</button>`
})
export class ExampleComponent {
  counter = 0;

  constructor(private ngZone: NgZone) {}

  startTimer() {
    this.ngZone.runOutsideAngular(() => {
      setInterval(() => {
        this.counter++; // This update will not trigger change detection
        console.log(`Counter: ${this.counter}`);
      }, 1000);
    });
  }
}
```

✅ **Improvement:** Now, the counter updates **without triggering change detection**, improving performance.

◆ **Bringing Execution Back to Angular (run())**

If you need to update the UI inside `NgZone`, use `.run()`.

Example: Re-entering Angular's Zone

```
this.ngZone.runOutsideAngular(() => {
  setTimeout(() => {
    this.ngZone.run(() => {
      this.counter++; // UI updates now
    });
  }, 1000);
});
```

🚀 **Best Practice:** Use `NgZone` **only for performance optimization** when dealing with background tasks!

Que. What is a Shared Module in Angular?

A **Shared Module** in Angular is a module that contains **reusable components, directives, pipes, and services** that can be used across multiple feature modules without duplication.

✦ Why Use a Shared Module?

✓ **Avoid Code Duplication** – Reuse common components like buttons, loaders, and UI elements.

✓ **Keep App Modular** – Separate shared functionality from feature modules.

✓ **Improve Maintainability** – Change once, update everywhere.

Note: In **Angular 16**, standalone components have removed the need for **NgModules**, making the concept of a **Shared Module** different. Instead of using NgModule, we **directly import and export components, directives, and pipes**.

Que. What is interface in angular?

In Angular, **interfaces** are used to define the structure of objects, ensuring type safety and consistency. They help enforce required properties and methods for objects, making the code more maintainable and reducing errors. Interfaces are commonly used with HTTP responses, components' inputs/outputs, and service methods for better type checking and clarity.

Que. What is used of app module

1. Global services (like singleton services) in the providers array so they are available throughout the app.
2. HTTP interceptors (for request/response handling) to globally manage HTTP operations.
3. Root component (typically AppComponent) that gets bootstrapped at app startup.
4. Feature modules that can be imported to be available globally in the application.

Example of AppModule:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, HttpClientModule], // Importing necessary modules
  providers: [
    AuthService, // Global services
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true } // Adding
    interceptor globally
  ],
  bootstrap: [AppComponent] // Bootstrapping the root component
})
export class AppModule {}
```

Why it's important:

- It acts as the entry point for bootstrapping Angular apps.
- You register global dependencies and configurations for the entire application.

Note: In Angular 18, standalone components handle bootstrapping, imports, and providers directly, eliminating the need for an AppModule. Global services and interceptors are configured within the component using the providers array.

Example:

```
import { Component } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';
import { AuthService } from './auth.service'; // Your AuthService
import { AuthInterceptor } from './auth.interceptor'; // Your AuthInterceptor
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [BrowserModule, HttpClientModule, RouterModule], // Importing necessary
  modules
  providers: [
    AuthService, // Global services
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true } // Adding
  interceptor globally
  ],
  template: `<h1>Welcome to Angular 18 Standalone!</h1>`
})
export class AppComponent {}
```

Que. How can we unsubscribe observables once move to next page?

1. ngOnDestroy
2. Async pipe

Que. How avoid multiple API calls on button clicks

1. **Disable Button:** Disable the button during the API call to prevent further clicks.
`<button [disabled]="isLoading" (click)="fetchData()">Fetch Data</button>`
2. **Use debounceTime or throttleTime:** Limit API calls to a certain time interval.

```
fromEvent(button, 'click')
  .pipe(
    debounceTime(500), // Wait 500ms after last click
    switchMap(() => this.apiService.getData())
  )
  .subscribe();
```
3. **Use switchMap:** Cancel ongoing API calls and only keep the latest one.

```
this.clickSubject.pipe(
  switchMap(() => this.apiService.getData())
).subscribe();
```
4. **Using loader or spinner**

Que. Difference between Cold Observable and Hot Observable

Cold Observable:

- Emits values only when subscribed.
- Each subscriber gets its own independent execution.
- Example: HTTP requests.

Hot Observable:

- Emits values regardless of subscription.
- Shares execution among all subscribers.
- Example: Subjects, WebSockets.

Que. Difference between Debouncing and Throttling

1. Debouncing:

- Executes a function after a specified delay, only if the event stops firing for a set amount of time.
- Ideal for scenarios like search input or window resize where you want to wait for the user to finish.
- Example: Wait for 300ms after typing stops before triggering an API call.

2. Throttling:

- Executes a function at a fixed interval, regardless of how many times the event occurs.
- Useful for high-frequency events like scrolling or mouse movement.
- Example: Trigger a function at most once every 500ms during scrolling.

Difference:

- **Debouncing** waits for the event to stop.
- **Throttling** limits the execution to a maximum frequency over time.



Que. What are Standalone Components in Angular?

Standalone Components in Angular are components that **do not require an NgModule**. They are **self-contained**, meaning they can import dependencies directly without being part of a module.

Cmd: ng g component example --standalone

Que. What are the key benefits of using Standalone Components?

- Eliminates unnecessary NgModules → Simplifies project structure.
- Reduces boilerplate code → No need to declare components in modules.
- Better tree shaking & optimization → Smaller bundle size.
- Easier lazy loading → Components can be loaded directly.
- Faster development → More modular and reusable.

Que. What Angular version introduced Standalone Components?

Standalone Components were introduced in **Angular 14** and became stable in **Angular 15**.

Que. 8. How do you import and use other components inside a Standalone Component?

Use the imports array:

```
import { CommonModule } from '@angular/common';
import { AnotherComponent } from './another.component';
```

```
@Component({
  selector: 'app-example',
  standalone: true,
  imports: [CommonModule, AnotherComponent],
  template: `<app-another></app-another>`
})
export class ExampleComponent {}
```

Que How do you bootstrap an Angular application using only a Standalone Component?

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app.component';
```

```
bootstrapApplication(AppComponent);
```

Que. Can Standalone Components work alongside traditional NgModules? If yes, how?

Yes. You can **import Standalone Components into an NgModule**, and vice versa.

Que. How do Standalone Components handle dependencies such as FormsModule and HttpClientModule?

Standalone Components can directly **import** modules:

```
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';
```

```
@Component({
  standalone: true,
  imports: [FormsModule, HttpClientModule],
  template: `<input [(ngModel)]="name">`
})
export class ExampleComponent {}
```

Que. Can you create Standalone Directives and Pipes? If yes, how?

Yes, by using standalone: true:

```
import { Directive } from '@angular/core';
```

```
@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective {}
```

Que. How do you lazy load a Standalone Component dynamically?

```
import { Routes } from '@angular/router';
```

```
const routes: Routes = [
  { path: 'example', loadComponent: () => import('./example.component').then(m =>
m.ExampleComponent) }
];
```

Que. How do you manage routing with Standalone Components?

```
import { provideRouter, Routes } from '@angular/router';
```

```
const routes: Routes = [{ path: '', component: ExampleComponent }];
```

```
bootstrapApplication(AppComponent, { providers: [provideRouter(routes)] });
```

Que. How do you configure providers inside a Standalone Component?

Use the providers array:

```
@Component({
  standalone: true,
  providers: [MyService]
})
export class ExampleComponent {}
```

Que. How do you migrate an existing Angular application to Standalone Components?

- Convert components to standalone: true.
- Remove them from NgModule.
- Import required dependencies in imports.

Que. What are the two types of forms in Angular?

1. Template-driven Forms:

- Definition: These forms are defined in the template (HTML) and use Angular directives like ngModel to create two-way data binding between the view and the model.

```
<form #myForm="ngForm">
  <input [(ngModel)]="user.name" name="name" required>
  <button [disabled]="!myForm.valid">Submit</button>
</form>
```

2. Reactive Forms:

- **Definition:** Reactive forms are defined programmatically in the component using classes like FormControl, FormGroup, and FormArray. These forms are more explicit and provide better control over form logic.

- ```
this.form = this.fb.group({
 name: ['', Validators.required]
});

<form [formGroup]="form">
 <input formControlName="name">
 <button [disabled]="!form.valid">Submit</button>
</form>
```

### Summary:

- **Template-driven forms** are simpler, declarative, and suitable for basic use cases.
- **Reactive forms** offer more control, flexibility, and scalability, especially for complex forms with dynamic data.

## Que. What is Used of ngForm

The ngForm directive in Angular is a special directive used in **template-driven forms** to track and manage the state of form elements and handle form submission

### Key Features of ngForm:

#### 1. Form State Management:

ngForm automatically tracks the following states:

- **valid:** Form is valid (all controls are valid).
- **invalid:** Form is invalid (some control is invalid).
- **dirty:** The form has been modified (some input changed).
- **pristine:** The form has not been modified.
- **touched:** The form has been visited by the user.
- **untouched:** The form has not been visited by the user.

```
<form #myForm="ngForm" (ngSubmit)="onSubmit()">
 <input [(ngModel)]="user.name" name="name" required #name="ngModel">
 <div *ngIf="name.invalid && name.touched">Name is required</div>

 <input [(ngModel)]="user.email" name="email" required email #email="ngModel">
 <div *ngIf="email.invalid && email.touched">Valid email is required</div>

 <button type="submit" [disabled]="!myForm.valid">Submit</button>
</form>
```

```
export class MyComponent { user = { name: '', email: '' }; onSubmit() { console.log('Form submitted', this.user); } }
```

### **Que. What are FormControl, FormGroup, and FormArray?**

**FormControl:** Represents an individual form control, like a text input.

**FormGroup:** A collection of FormControl or other FormGroup instances that represent a section of the form.

**FormArray:** Manages an array of FormControl or FormGroup instances, typically used when the number of controls is dynamic.(eg. Add multiple mobile no. dynamically)

### **Que. How do you perform validation in Angular forms?**

- **Template-driven validation:** You use validators like required, minlength, etc., directly in the HTML template.
- **Reactive form validation:** You use validators in the component class, such as Validators.required, Validators.minLength, Validators.email, etc., while defining the form controls.

### **Que. How do you reset a form in Angular?**

- You can reset a form by calling the reset() method on the form group or form control: `this.form.reset();`

### **What is the role of FormBuilder in Angular?**

- FormBuilder is a service that simplifies the creation of form controls, form groups, and form arrays.
- It provides convenient methods like group(), control(), and array() for defining reactive forms in a more readable and concise way.

```
this.form = this.fb.group({
 name: ['', Validators.required],
 email: ['', [Validators.required, Validators.email]]
});
```

### **Que. How do you dynamically add or remove form controls in Angular?**

- In reactive forms, you can dynamically add or remove form controls using methods like addControl() and removeControl():

```
this.form.addControl('newField', new FormControl(''));
this.form.removeControl('oldField');
```

### **Que. How do you use patchValue() and setValue() in reactive forms?**

- **setValue():** Used to set the entire value of the form group or form control. It requires an object with values for all controls.

```
this.form.setValue({
 firstName: 'John',
 lastName: 'Doe'
});
```

- **patchValue():** Used to update part of the form value. It doesn't require all values to be provided.

```
this.form.patchValue({
 firstName: 'John'
});
```

#### **Que. How do you track form state changes in Angular?**

- You can track the form's state (pristine, touched, dirty, valid, invalid) by using the properties of FormControl or FormGroup:

```
console.log(this.form.invalid); // Check if form is invalid
console.log(this.form.pristine);
```

#### **Que. How do you display error messages for form validation in Angular?**

- In template-driven forms, you can use ngIf to conditionally display error messages:

```
<div *ngIf="form.controls['firstName'].invalid && form.controls['firstName'].touched">
 First Name is required.
</div>
```

#### **Que. How do you create a multi-step form using Angular?**

- You can manage a multi-step form by creating multiple FormGroup instances, each corresponding to a different step of the form. For each step, the form controls are updated, validated, and submitted accordingly.

#### **Que. Creating a Signup Form in Angular with Password Validation and Value Restriction (Less than 0 and More than 100)**

##### **Step 1: Setup the TypeScript Component**

First, we need to import the necessary modules and set up the form in our component.

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
@Component({
 selector: 'app-signup',
 templateUrl: './signup.component.html',
 styleUrls: ['./signup.component.css']
})
export class SignupComponent {
```

```
 // Create FormGroup for Signup form
 signupForm = new FormGroup({
 // FormControl for name (required)
 name: new FormControl('', [Validators.required]),
```

```
 // FormControl for email (required and email format)
 email: new FormControl('', [Validators.required, Validators.email]),
```

```

// FormControl for password with custom validation
password: new FormControl("", [
 Validators.required,
 Validators.minLength(8),
 this.passwordValidator
]),

// FormControl for age with custom number restriction
age: new FormControl("", [
 Validators.required,
 Validators.min(1),
 Validators.max(100)
]),
});

// Custom password validator (checks for at least one uppercase letter, one number, and a
minimum length of 8)
passwordValidator(control: FormControl) {
 const value = control.value;
 if (value && ![A-Z]/.test(value)) {
 return { 'passwordStrength': 'Password must contain at least one uppercase letter' };
 }
 if (value && !/\d/.test(value)) {
 return { 'passwordStrength': 'Password must contain at least one number' };
 }
 return null;
}

// Function to handle form submission
onSubmit() {
 if (this.signupForm.valid) {
 console.log('Form Submitted', this.signupForm.value);
 } else {
 console.log('Form is invalid');
 }
}
}

```

## Step 2: Apply Validators

- **Email Field:** The email field is validated with the `Validators.email` and `Validators.required` to ensure it contains a valid email address and is not empty.
- **Password Field:** Custom validators are applied to check if the password contains:
  - At least one uppercase letter.
  - At least one digit.

- A minimum length of 8 characters.
- **Age Field:** The age is validated to ensure the value is between 1 and 100 using the `Validators.min()` and `Validators.max()` functions.

### Step 3: HTML Template

Now, let's create the form in the **HTML** template to display the input fields for the user.

```
<div class="container">
 <form [formGroup]="signupForm" (ngSubmit)="onSubmit()">

 <!-- Name Input -->
 <div class="form-group">
 <label for="name">Name</label>
 <input type="text" id="name" formControlName="name" class="form-control"
placeholder="Enter your name" />
 <div *ngIf="signupForm.get('name').invalid && signupForm.get('name').touched"
class="text-danger">
 Name is required
 </div>
 </div>

 <!-- Email Input -->
 <div class="form-group">
 <label for="email">Email</label>
 <input type="email" id="email" formControlName="email" class="form-control"
placeholder="Enter your email" />
 <div *ngIf="signupForm.get('email').invalid && signupForm.get('email').touched"
class="text-danger">
 Enter a valid email
 </div>
 </div>

 <!-- Password Input -->
 <div class="form-group">
 <label for="password">Password</label>
 <input type="password" id="password" formControlName="password" class="form-
control" placeholder="Enter your password" />
 <div *ngIf="signupForm.get('password').invalid && signupForm.get('password').touched"
class="text-danger">
 <div *ngIf="signupForm.get('password').errors?.['passwordStrength']">
 {{ signupForm.get('password').errors?.['passwordStrength'] }}
 </div>
 <div *ngIf="signupForm.get('password').hasError('minlength')">
 Password must be at least 8 characters long
 </div>
 </div>
 </div>
```

```

</div>

<!-- Age Input -->
<div class="form-group">
 <label for="age">Age</label>
 <input type="number" id="age" formControlName="age" class="form-control"
placeholder="Enter your age" />
 <div *ngIf="signupForm.get('age').invalid && signupForm.get('age').touched"
class="text-danger">
 Age must be between 1 and 100
 </div>
</div>

<!-- Submit Button -->
<button type="submit" class="btn btn-primary"
[disabled]="signupForm.invalid">Submit</button>
</form>
</div>

```

#### Que. How to optimize Angular app performance

1. **Lazy Loading:** Load modules only when needed, reducing the initial bundle size.
2. **Ahead-of-Time (AOT) Compilation:** Pre-compile templates and components for faster rendering.
3. **Tree Shaking:** Remove unused code during the build process to reduce bundle size.
4. **Change Detection Strategy:** Use ChangeDetectionStrategy.OnPush to optimize change detection by reducing checks.
5. **Minification and Bundling:** Minify and bundle JavaScript files to reduce file size and HTTP requests.
6. **Optimize Images:** Compress and serve images in modern formats (like WebP) for faster load times.
7. **Web Workers:** Offload heavy computations to background threads, keeping the UI responsive.
8. **TrackBy in ngFor:** Use trackBy to optimize the rendering of lists, reducing DOM updates.

#### Que. How to implement security measures for Angular

1. **Use HTTPS:** Ensure all communications between client and server are encrypted to prevent man-in-the-middle attacks.
2. **Sanitize User Input:** Angular automatically sanitizes templates to prevent XSS attacks. Use DomSanitizer for raw HTML when necessary.
3. **CSRF Protection:** Implement CSRF tokens and use SameSite cookies to prevent cross-site request forgery.
4. **Content Security Policy (CSP):** Enforce a CSP header to restrict the sources of scripts, images, and other resources, mitigating XSS risks.

5. **Secure Storage:** Avoid storing sensitive data in localStorage or sessionStorage; use HttpOnly cookies for secure token storage.
6. **Prevent Clickjacking:** Set the X-Frame-Options header to DENY to prevent embedding your site in iframes.
7. **JWT Authentication:** Use JWTs for secure user authentication. Store them in HttpOnly cookies and secure them via HTTPS.
8. **Route Guards:** Protect routes with Angular route guards to ensure users can only access pages they're authorized for.
9. **File Upload Security:** Validate file types and sizes before uploading and ensure server-side validation as well.
10. **Strong Password Policies:** Implement strong password validation using Angular forms to enforce secure user passwords.

What is the difference between bootstrapApplication() and bootstrapModule()?

What is lazy Loading in Angular?